

Insitut National des Postes et Telecommunications

Federated learning and SLM
Rapport

Realisé par :
Zakaria Bigane
Marwan Moujane

Encadrant :
Dr. Meryem Ayache & Dr.
Douae El Achhab

15 juin 2026

Table des matières

1 Introduction et État de l'Art **4**
1.1 État de l'art sur la sécurité du code généré par LLM 4
1.2 Apprentissage fédéré pour la confidentialité des données 4
1.3 Pourquoi des Petits Modèles de Langage (SLM) ? 5
1.4 Apprentissage Fédéré : Principes Mathématiques de FedAvg et FedProx . . 5
1.5 Objectifs et contributions du projet 6

2 Partie 1 : Ajustement Centralisé d'un Petit Modèle de Langue **7**
2.1 Choix du Modèle et Optimisation pour Ressources Contraintes 7
2.2 Préparation du Jeu de Données Unifié 8
2.3 Mise en Place de l'Entraînement avec LoRA et SFTTrainer 9
2.4 Suivi de l'Entraînement et Évolution de la Loss 11
2.5 Validation Qualitative : Génération et Détection de Code Sûr 11
2.6 Évaluation Quantitative des Performances de Détection 14
2.7 Évaluation Quantitative des Performances de Génération 16
2.8 Sauvegarde et Fusion du Modèle 18

3 Partie 2 : Simulation d'Apprentissage Fédéré **19**
3.1 Motivation et Architecture 19
3.2 Configuration de l'Environnement Fédéré 19
3.3 Moteur de Partitionnement Non-I.I.D. Dynamique 20
3.4 Implémentation du Client Virtuel 22
3.5 Orchestration et Agrégation 27
3.6 Déroulement et Résultats de la Simulation 29
3.7 Évaluation du Modèle Fédéré 30
3.8 Synthèse et Discussion : Analyse Comparée des Métriques et Interprétation des Résultats 34
3.8.1 Métriques de Détection Binaire : Explication Détaillée d'une Performance Proche du Hasard 34
3.8.2 Métriques de Génération : Explication Détaillée et Capacités Partiellement Préservées 36
3.8.3 Mise en Relation des Métriques et Discussion Globale 38
3.8.4 Perspectives d'Amélioration 38

4 Partie 3 : Déploiement Réel sur Raspberry Pi (Apprentissage Fédéré Physique) **40**
4.1 Transition de la Simulation vers le Matériel Réel 40
4.2 Architecture et Matériel 40
4.3 Jeux de Données Locaux et Respect de la Confidentialité 41
4.4 Implémentation des Clients Physiques (`client.py`) 43
4.5 Implémentation du Serveur Central (`server.py`) 46
4.6 Protocole Expérimental et Configurations 47
4.7 Résultats et Analyse des Performances 48
4.8 Synthèse et Comparaison avec les Parties Précédentes 52
4.9 Limites Pratiques du Déploiement sur Matériel Embarqué 53
4.10 Conclusion sur le Déploiement Réel 54

5 Conclusion **55**

A Annexe : Commandes d'Installation et Configuration Détaillée **57**
A.1 Environnement de développement centralisé (Kaggle / Colab) 57
A.2 Simulation d'apprentissage fédéré (Flower sur Kaggle) 57
A.3 Déploiement sur Raspberry Pi (ARM64) 57
A.4 Lancement des clients et du serveur 57
A.5 Configuration des hyperparamètres pour le déploiement réel 58
A.6 Jeux de Données Locaux Complets 58

Résumé

Ce projet de recherche s'inscrit dans le cadre de la sécurisation des assistants de programmation basés sur l'intelligence artificielle. Il explore le développement d'un pipeline complet allant de l'ajustement supervisé d'un Petit Modèle de Langage (SLM) pour la génération et la détection de code vulnérable, jusqu'à son déploiement fédéré sur des nœuds périphériques (cartes Raspberry Pi). L'objectif principal est de fournir une solution légère, respectueuse de la vie privée et adaptable à des environnements sensibles tels que les secteurs bancaire et médical.

La première phase consiste à fine-tuner de manière centralisée le modèle `Qwen2.5-Coder-0.5B-Instr` (500 millions de paramètres) à l'aide de la bibliothèque `Unsloth` et de la méthode d'adaptation paramétrique LoRA (rang 16). Un jeu de données unifié de 28 467 flashcards conversationnelles est construit à partir du corpus `StonyBrookNLP/DiSCo`, combinant des tâches de génération de code sécurisé (requêtes SQL paramétrées, fermeture de connexions) et de détection de vulnérabilités (audit de code vulnérable avec raisonnement explicite). L'entraînement sur GPU Tesla T4 atteint une loss de 0,296 après une époque. Le modèle unifié est évalué sur deux axes complémentaires. Pour la *détection de vulnérabilités*, une évaluation binaire sur 200 flashcards inédites donne une exactitude de 74,0%, une précision de 72,2%, un rappel de 78,0% et un F1-score de 75,0%. Pour la *génération de code sécurisé*, des métriques NLP de référence sont calculées sur un ensemble de validation indépendant de 100 exemples : une perplexité (PPL) de 1,80 (excellente fluence), un score SacreBLEU de 47,42, un CodeBLEU de 49,25 (correspondance syntaxique 58%, flot de données 45%) et un ROUGE-L de 55,95.

La deuxième phase introduit l'apprentissage fédéré via le framework
textttFlower (stratégie
textttFedProx) en simulant 10 clients virtuels (banques et hôpitaux). Par rapport à la version initiale, une amélioration architecturale majeure a été apportée : le remplacement du partitionnement i.i.d. par un moteur de partitionnement
emphnon-i.i.d. dynamique et réaliste. Chaque client reçoit des données skewées selon son profil métier — spécialiste génération (80

La troisième phase concrétise le passage au réel en déployant deux Raspberry Pi 4 (équipes Banque et Santé) communicant avec un serveur central via le protocole Flower. Quatre configurations hyperparamétriques sont testées, la meilleure (longueur de séquence 1028 tokens, rang LoRA 8, taux d'apprentissage 5×10^{-2}) permettant d'atteindre une précision globale (token-level) de 70,9 % après trois rondes, avec une loss réduite à 1,64. Ce résultat, obtenu sur des jeux de données locaux volontairement réduits (6 échantillons par client), démontre la viabilité d'un assistant de codage sécurisé, souverain et décentralisé, adapté aux infrastructures contraintes.

1 Introduction et État de l'Art

L'intégration des grands modèles de langage (LLMs) dans les chaînes de développement logiciel a considérablement amélioré la productivité des développeurs. Des outils comme GitHub Copilot ou ChatGPT permettent désormais de générer automatiquement du code, de compléter des fonctions ou même d'expliquer des extraits complexes. Cependant, cette automatisation s'accompagne de risques majeurs : les LLMs produisent fréquemment du code contenant des vulnérabilités de sécurité, allant des injections SQL aux débordements de mémoire, en passant par une mauvaise gestion des accès ou des faiblesses cryptographiques. Des études récentes estiment que plus de 75 % du code généré par des modèles comme GPT-3.5 ou GPT-4 contient au moins une faille exploitable [**gong2025well**]. Cette réalité compromet la confiance que l'on pourrait accorder aux assistants de programmation basés sur l'IA.

1.1 État de l'art sur la sécurité du code généré par LLM

De nombreuses recherches ont mis en lumière l'ampleur du problème. La revue systématique de Basic et Giaretta [**basic2025vulnerabilities**] recense les vulnérabilités les plus fréquentes : injections (SQL, XSS, commandes OS), erreurs de gestion mémoire (déréférencement de pointeur nul, use-after-free), exposition de données sensibles, faiblesses d'authentification ou d'autorisation, et mauvaises pratiques cryptographiques. Face à ces menaces, plusieurs approches de détection et de correction ont été proposées.

Zhang et al. [**zhang2025code**] combinent analyse statique (Bandit, CodeQL) et fuzzing dynamique (AFL++) pour détecter et corriger les vulnérabilités dans le code généré par LLM, obtenant une réduction de 68,2 % des failles. Nunez et al. [**nunez2024autosafecoder**] proposent une architecture multi-agents (AutoSafeCoder) où un agent de génération interagit avec un analyseur statique et un agent de fuzzing pour améliorer itérativement la sécurité du code, atteignant une réduction de 13 % des vulnérabilités par rapport à GPT-4 seul. Gong et al. [**gong2025well**] montrent que les LLMs souffrent d'« angles morts d'auto-réparation » : ils réparent moins efficacement leur propre code que celui d'autres modèles. Yan et Zhang [**yan2025guiding**] démontrent l'intérêt d'indices de vulnérabilité contextualisés (générés par le modèle lui-même) pour prévenir les failles lors de la génération.

Parallèlement, une tendance émergente consiste à utiliser des Petits Modèles de Langage (SLMs), plus économes en ressources et faciles à déployer localement. Bappy et al. [**bappy2025casestudy**] fine-tunent un modèle CodeGen-Mono (350M paramètres) pour la détection CWE en Python, atteignant 99 % de précision. Kim et Noller [**kim2024codexity**] intègrent des analyseurs statiques (CppCheck, Infer) à des SLMs pour réduire de 60 % la génération de code vulnérable. Hasan et al. [**hasan2026assessing**] comparent 20 SLMs (0,4 B à 10 B paramètres) sur cinq benchmarks et montrent que certains modèles compacts offrent des performances compétitives avec une consommation mémoire bien moindre.

1.2 Apprentissage fédéré pour la confidentialité des données

Dans les secteurs sensibles (banque, santé, défense), les données de code (logs de projets, extraits de bases de données propriétaires) ne peuvent pas être transmises à des serveurs cloud pour entraîner des modèles. L'apprentissage fédéré (Federated Learning)

répond à cette contrainte en permettant à plusieurs clients d'entraîner collaborativement un modèle sans partager leurs données brutes : seuls les gradients ou les poids des couches d'adaptation (LoRA) sont échangés avec un serveur central. La plateforme Flower facilite la mise en œuvre de tels systèmes, avec des stratégies d'agrégation comme FedAvg ou FedProx (qui ajoute un terme de proximité pour stabiliser la convergence en présence de données non i.i.d.).

1.3 Pourquoi des Petits Modèles de Langage (SLM) ?

Le choix de travailler avec des SLMs plutôt qu'avec des modèles de plusieurs dizaines de milliards de paramètres répond à des contraintes très concrètes de déploiement. Un modèle de 0,5 milliard de paramètres, une fois quantifié, occupe moins de 600 Mo de RAM, ce qui le rend exploitable sur un Raspberry Pi 4 disposant de 4 Go de mémoire, sur une tablette professionnelle ou même sur un poste de travail sans GPU dédié. Cette caractéristique est essentielle pour les scénarios fédérés visés par ce projet : une banque ou un hôpital ne dispose pas nécessairement de fermes de GPU pour entraîner un modèle de 70 milliards de paramètres en local, mais peut tout à fait consacrer une carte embarquée à un agent de codage sécurisé qui tourne en permanence sur le réseau interne.

De plus, les SLMs spécialisés dans le code (Qwen2.5-Coder, CodeGen, StarCoder-mini, etc.) bénéficient d'un pré-entraînement massif sur des corpus de code source, ce qui leur confère déjà une bonne compréhension syntaxique et sémantique des langages de programmation avant même le fine-tuning. Le fine-tuning par LoRA permet alors de *spécialiser* ce socle de connaissances sur deux tâches précises (génération sécurisée et détection de vulnérabilités) sans avoir à ré-entraîner l'ensemble du réseau, ce qui réduit considérablement le coût computationnel et le risque de "catastrophic forgetting" (oubli catastrophique des capacités générales du modèle).

1.4 Apprentissage Fédéré : Principes Mathématiques de FedAvg et FedProx

Pour bien comprendre les résultats obtenus dans les parties 2 et 3, il est utile de revenir brièvement sur le fonctionnement mathématique des deux stratégies d'agrégation utilisées.

FedAvg. La stratégie de base, `Federated Averaging`, consiste, à chaque ronde t, à envoyer le modèle global w_t à un sous-ensemble de clients K. Chaque client k effectue E époques d'entraînement local sur ses données privées D_k pour obtenir des poids locaux w_{t+1}^k. Le serveur agrège ensuite ces poids par une moyenne pondérée par la taille des jeux de données locaux :

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k \tag{1}$$

où $n_k = |D_k|$ est le nombre d'exemples du client k et $n = \sum_k n_k$ le total. Cette simplicité fait de FedAvg la stratégie par défaut pour notre déploiement réel sur Raspberry Pi (Partie 3), où seulement deux clients participent et où la convergence rapide est prioritaire.

FedProx. Lorsque les données entre clients sont fortement hétérogènes (non-i.i.d.), FedAvg peut diverger ou converger lentement, car chaque client peut "trop s'éloigner" du modèle global pendant son entraînement local. FedProx [**li2020federated**] introduit un terme de régularisation proximale dans la fonction de perte locale :

$$\min_{w} \; F_k(w) + \frac{\mu}{2}\|w - w_t\|^2 \tag{2}$$

où $F_k(w)$ est la fonction de perte locale du client k, w_t sont les poids globaux reçus en début de ronde, et μ est un coefficient de proximité (fixé à $0,01$ dans notre simulation, Partie 2). Ce terme pénalise les mises à jour locales qui dérivent trop loin du modèle global, ce qui stabilise la convergence dans des contextes hétérogènes, au prix d'une convergence parfois plus lente lorsque μ est mal calibré ou que le nombre de rondes est trop faible — un point que nous discuterons en détail dans la synthèse de la Partie 2.

1.5 Objectifs et contributions du projet

Notre projet s'inscrit dans cette dynamique en proposant une approche unifiée couvrant l'ensemble du cycle de vie d'un assistant de codage sécurisé et fédéré :

1. **Ajustement centralisé** d'un SLM (Qwen2.5-Coder-0.5B) pour qu'il sache à la fois générer du code sécurisé (requêtes paramétrées, fermeture de ressources) et auditer du code existant en identifiant les vulnérabilités avec un raisonnement explicite.
2. **Simulation d'apprentissage fédéré** avec 10 clients virtuels (banques et hôpitaux) pour évaluer la faisabilité et les limites de l'approche en environnement i.i.d. avec peu de rondes.
3. **Déploiement réel sur Raspberry Pi** (deux nœuds physiques) avec des jeux de données locaux non i.i.d., démontrant la viabilité d'un système décentralisé, respectueux de la vie privée, fonctionnant sur du matériel contraint.

Les résultats montrent que le modèle centralisé atteint 74,0

2 Partie 1 : Ajustement Centralisé d'un Petit Modèle de Langue

2.1 Choix du Modèle et Optimisation pour Ressources Contraintes

Le modèle retenu est `Qwen2.5-Coder-0.5B-Instruct`, un petit modèle de langue (SLM) de 500 millions de paramètres, spécifiquement entraîné pour les tâches de programmation. Il offre un excellent compromis entre performance et faible empreinte mémoire, ce qui le rend idéal pour un futur déploiement sur des cartes Raspberry Pi ou d'autres périphériques à ressources limitées.

Afin de réduire la consommation de mémoire vive (VRAM) et d'accélérer l'entraînement, nous utilisons la bibliothèque `Unsloth` (version 2026.6.1) qui propose des implémentations optimisées des couches Transformer (notamment pour l'architecture Qwen2). Le chargement du modèle est effectué avec l'option `load_in_4bit = True`, ce qui quantifie les poids en virgule flottante 4 bits et divise par quatre la mémoire nécessaire. La longueur de séquence maximale est fixée à 1024 tokens, ce qui couvre largement la taille des extraits de code manipulés.

L'adaptation paramétrique repose sur la méthode `LoRA` (Low-Rank Adaptation) implémentée dans la bibliothèque `PEFT`. Nous configurons un rang de 16 et un coefficient `lora_alpha` également égal à 16, avec un dropout nul. Les modules ciblés sont les projections des requêtes, des clés, des valeurs et de la sortie (`q_proj`, `k_proj`, `v_proj`, `o_proj`), ainsi que les trois projections des couches MLP (`gate_proj`, `up_proj`, `down_proj`). Cette configuration ne rend entraînables que 8,8 millions de paramètres (soit 1,75 % du total de 502,8 millions), ce qui réduit considérablement la charge mémoire et le temps de calcul.

```
    from unsloth import FastLanguageModel
    import torch
    from datasets import Dataset

    print("Loading the Qwen Model into the GPU (1024 Context)...")
    model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Qwen2.5-Coder-0.5B-Instruct",
    max_seq_length = 1024,
    dtype = None,
    load_in_4bit = True,
    )

    print("Applying LoRA adapters...")
    model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
    "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 3407,
    )

```

Listing 1 – Chargement du modèle quantifié et application de LoRA avec Unsloth

L'option `use_gradient_checkpointing = "unsloth"` mérite une attention particulière : il s'agit d'une implémentation propriétaire d'Unsloth qui décale le recalcul des activations intermédiaires de manière asynchrone, ce qui permet de réduire encore davantage l'empreinte mémoire par rapport au gradient checkpointing standard de PyTorch, sans pénaliser significativement la vitesse d'entraînement. Le paramètre `random_state = 3407` fixe la graine aléatoire pour assurer la reproductibilité de l'initialisation des matrices LoRA (A et B), un détail important pour comparer équitablement différentes exécutions.

2.2 Préparation du Jeu de Données Unifié

Le jeu de données source est `StonyBrookNLP/DiSCo` (fichier `DiSCo_train.csv`), qui contient 9 489 exemples. Chaque exemple comporte quatre champs : une instruction de génération, un code vulnérable, le code corrigé sécurisé et un raisonnement textuel expliquant la vulnérabilité.

Nous construisons un jeu de données unifié en générant trois types de "flashcards conversationnelles" au format ChatML :

1. **Flashcard de génération** : l'instruction utilisateur est le prompt de génération original ; la réponse de l'assistant est le code sécurisé.
2. **Flashcard de détection (code vulnérable)** : l'utilisateur demande d'auditer le code ; l'assistant répond "VULNÉRABLE." suivi du raisonnement extrait du jeu de données.
3. **Flashcard de détection (code sécurisé)** : l'utilisateur demande d'auditer le code sécurisé ; l'assistant répond "SÉCURISÉ. Ce code suit les bonnes pratiques et ne contient pas de vulnérabilités évidentes."

La conversion utilise la fonction `apply_chat_template` du tokenizer avec `add_generation_prompt=F` pour stocker la conversation complète dans un champ `text`. Ce procédé étend le jeu de données à 28 467 exemples, permettant au modèle d'apprendre simultanément à générer du code sûr et à détecter des failles dans du code existant. Le Listing 2 présente l'implémentation complète de cette transformation.

```
    from datasets import load_dataset

    print("Downloading the DiSCo Secure Code Generation dataset...")
    dataset = load_dataset("StonyBrookNLP/DiSCo", data_files="
DiSCo_train.csv", split="train")
    print(f"Success! Loaded {len(dataset)} generation examples into
memory.")

    print("Creating UNIFIED dataset (generation + detection flashcards)
...")
    unified_data = []

    for row in dataset:
    instruction = str(row.get("Instruction", ""))
    secure_code = str(row.get("Secure Code", ""))
    vuln_code   = str(row.get("Vulnerable Code", ""))
    reasoning   = str(row.get("Reasoning", ""))

    # 1. Flashcard de generation
    gen_convo = [
```

```
    {"role": "user", "content": instruction},
    {"role": "assistant", "content": secure_code}
    ]
    gen_text = tokenizer.apply_chat_template(gen_convo, tokenize=False,
    add_generation_prompt=False)
    unified_data.append({"text": gen_text})

    # 2. Flashcard de detection (code vulnerable)
    if vuln_code and reasoning:
    vuln_convo = [
    {"role": "user", "content": f"Audit this Python code for
vulnerabilities:\n\n{vuln_code}"},
    {"role": "assistant", "content": f"VULNERABLE.\nReasoning: {
reasoning}"}
    ]
    vuln_text = tokenizer.apply_chat_template(vuln_convo, tokenize=False
,
    add_generation_prompt=False)
    unified_data.append({"text": vuln_text})

    # 3. Flashcard de detection (code securise)
    if secure_code:
    secure_convo = [
    {"role": "user", "content": f"Audit this Python code for
vulnerabilities:\n\n{secure_code}"},
    {"role": "assistant", "content": "SECURE. This code follows best
practices and contains no obvious vulnerabilities."}
    ]
    secure_text = tokenizer.apply_chat_template(secure_convo, tokenize=
False,
    add_generation_prompt=False)
    unified_data.append({"text": secure_text})

    unified_dataset = Dataset.from_list(unified_data)
    print(f"Original rows: {len(dataset)}")
    print(f"Unified training examples: {len(unified_dataset)} (
generation + detection flashcards)")

```

Listing 2 – Construction du jeu de données unifié (génération + détection)

Ce procédé d'augmentation de données possède une propriété intéressante : pour chaque ligne du jeu de données original (9 489 lignes), trois exemples conversationnels sont générés (lorsque les champs requis sont non vides), ce qui explique le facteur de multiplication d'environ 3 et le total de 28 467 exemples ($9489 \times 3 \approx 28467$). Cette stratégie permet au modèle de voir, pour un même extrait de code, à la fois la version sécurisée comme *cible de génération* et comme *exemple positif d'audit*, tandis que la version vulnérable est présentée comme *exemple négatif d'audit* accompagné d'un raisonnement explicite. Cette triple perspective (générer, auditer du bon code, auditer du mauvais code) renforce la cohérence interne du modèle entre ses deux tâches.

2.3 Mise en Place de l'Entraînement avec LoRA et SFTTrainer

L'entraînement supervisé est réalisé avec `SFTTrainer` de la bibliothèque `trl` (Hugging Face). Les hyperparamètres sont choisis pour s'adapter à la mémoire limitée d'un GPU

Tesla T4 (environ 15 Go) :

- `per_device_train_batch_size = 1` : une seule séquence par lot.
- `gradient_accumulation_steps = 8` : taille de lot effective de 8.
- `num_train_epochs = 1` : une époque complète, soit 1 780 étapes.
- `learning_rate = 2e-4` avec un scheduler linéaire et 10 étapes de warmup.
- Optimiseur `adamw_8bit` (bitsandbytes) pour réduire la consommation mémoire.
- Précision mixte BF16 (activée via `bf16 = True`) car le T4 supporte BF16.
- `packing = False` pour éviter de concaténer des séquences de longueurs très variables.

La fonction de perte est l'entropie croisée classique pour la modélisation de langage causal. Le Listing 3 illustre la configuration complète de l'entraîneur.

```
    from trl import SFTTrainer
    from transformers import TrainingArguments
    from unsloth import is_bfloat16_supported

    trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = unified_dataset,
    dataset_text_field = "text",
    max_seq_length = 1024,
    dataset_num_proc = 1,
    packing = False,
    args = TrainingArguments(
    per_device_train_batch_size = 1,
    gradient_accumulation_steps = 8,
    warmup_steps = 10,
    num_train_epochs = 1,
    learning_rate = 2e-4,
    fp16 = not is_bfloat16_supported(),
    bf16 = is_bfloat16_supported(),
    logging_steps = 10,
    optim = "adamw_8bit",
    weight_decay = 0.01,
    lr_scheduler_type = "linear",
    seed = 3407,
    output_dir = "outputs",
    report_to = "none",
    average_tokens_across_devices = False,
    save_strategy = "no",
    ),
    )

    trainer_stats = trainer.train()

```

Listing 3 – Configuration de l'entraîneur SFT avec Unsloth

Le choix d'un `batch size` effectif de 8 (1×8 accumulations) résulte d'un compromis entre stabilité de la descente de gradient et contrainte mémoire. Avec une longueur de séquence maximale de 1024 tokens et un modèle de 0,5 milliard de paramètres en 4-bit, un batch size physique supérieur à 1 provoquait des erreurs `CUDA out of memory` sur le GPU Tesla T4 (15 Go). L'accumulation de gradient permet de simuler un batch plus large sans dépasser la mémoire disponible, au prix d'un temps d'entraînement légèrement plus

long.

2.4 Suivi de l'Entraînement et Évolution de la Loss

Au cours des 1 780 étapes, la loss d'entraînement diminue progressivement. La Table 1 présente quelques valeurs représentatives. La loss initiale aux alentours de 2,23 chute à 0,313 à la fin de l'époque, confirmant que le modèle converge correctement.

TABLE 1 – Évolution de la loss d'entraînement (valeurs moyennes).

Étape	Loss
10	2,231
100	0,630
500	0,438
1000	0,356
1500	0,312
1780	0,314

On observe que la courbe de loss présente une décroissance très rapide durant les 100 premières étapes (de 2,231 à 0,630), correspondant à la phase où le modèle apprend rapidement le format ChatML et la structure générale attendue des réponses (code Python, balises "VULNERABLE."/"SECURE.", etc.). Au-delà de l'étape 500, la décroissance devient beaucoup plus lente, ce qui est typique d'un fine-tuning LoRA : les couches de bas rang ajustent finement le comportement du modèle sans modifier fondamentalement ses représentations internes héritées du pré-entraînement. La quasi-stagnation entre les étapes 1500 et 1780 (0,312 à 0,314) suggère que le modèle a atteint un plateau de convergence pour une seule époque, et qu'un entraînement plus long apporterait probablement des gains marginaux de plus en plus faibles, avec un risque accru de sur-apprentissage (overfitting) sur les motifs syntaxiques spécifiques au corpus DiSCo.

2.5 Validation Qualitative : Génération et Détection de Code Sûr

Après l'entraînement, nous évaluons le modèle sur cinq prompts de génération (requêtes SQL paramétrées) et cinq extraits de code vulnérable (injections SQL). Le Listing 4 présente l'ensemble du protocole de test qualitatif, incluant les cinq formulations différentes du même besoin fonctionnel (récupération du profil utilisateur) et les cinq extraits de code volontairement vulnérables soumis à l'audit.

```
1    from transformers import TextStreamer
2
3    # ----- 5 Prompts de generation (variations) -----
4    gen_prompts = [
5    "Write a Python function that accepts a username and retrieves the
     user "
6    "profile from a SQLite database. Use parameterized queries to
     prevent "
7    "SQL injection. Return the profile as a dictionary or None if not
     found.",
8
9    "Implement a Python function called get_user_profile that connects
     to "
```

```
    "users.db, queries the users table by username, and returns the row as "
    "a dict. Include basic error handling.",

    "Python function: SQLite query to fetch user profile by username. "
    "Use safe query practices and close the connection.",

    "Given a SQLite database with a users table (id, username, email, "
    "created_at), write a function that takes a username string and returns "
    "the matching row as a dict. Return None if not found. No ORM.",

    "Write a secure Python function to look up a user profile from SQLite "
    "by username. Must use parameterized queries, close the connection, "
    "and return a dict or None."
    ]

    # ----- 5 extraits de code vulnerable a auditer -----
    vuln_code_examples = [
    """def get_user(username):
    conn = sqlite3.connect('db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM users WHERE name = '{username}'")
    return cursor.fetchone()""",

    """def login(user, pwd):
    query = "SELECT * FROM users WHERE user = '" + user + "' AND pass = '" + pwd + "'"
    cursor.execute(query)
    return cursor.fetchone()""",

    """def get_profile(user_id):
    conn = sqlite3.connect('app.db')
    cur = conn.cursor()
    cur.execute("SELECT * FROM profiles WHERE id = " + user_id)
    return cur.fetchall()""",

    """def search_products(keyword):
    conn = sqlite3.connect('shop.db')
    c = conn.cursor()
    c.execute("SELECT * FROM products WHERE name LIKE '%" + keyword + "%'")
    return c.fetchall()""",

    """def update_email(username, new_email):
    conn = sqlite3.connect('users.db')
    cur = conn.cursor()
    cur.execute("UPDATE users SET email = '" + new_email + "' WHERE username = '" + username + "'")
    conn.commit()
    return True"""
    ]

    text_streamer = TextStreamer(tokenizer, skip_prompt=True)

```

```
    # Test 1-5 : generation
    for i, prompt in enumerate(gen_prompts, 1):
    messages = [{"role": "user", "content": prompt}]
    formatted_prompt = tokenizer.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
    )
    inputs = tokenizer([formatted_prompt], return_tensors="pt").to("cuda
    ")
    _ = model.generate(**inputs, streamer=text_streamer,
    max_new_tokens=300, use_cache=False)

    # Test 6-10 : detection
    for i, vuln_code in enumerate(vuln_code_examples, 1):
    audit_prompt = f"Audit this Python code for vulnerabilities:\n\n{
    vuln_code}"
    messages = [{"role": "user", "content": audit_prompt}]
    formatted_prompt = tokenizer.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
    )
    inputs = tokenizer([formatted_prompt], return_tensors="pt").to("cuda
    ")
    _ = model.generate(**inputs, streamer=text_streamer,
    max_new_tokens=200, use_cache=False)

```

Listing 4 – Protocole de test qualitatif (5 generations + 5 audits)

Génération. Pour chaque prompt, le modèle produit systématiquement du code utilisant des requêtes paramétrées (caractère " ?" dans `cursor.execute`) et ferme les connexions SQLite. La réponse inclut également une gestion d'erreur basique lorsque le prompt le demande. Un exemple de sortie est donné dans le Listing 5.

```
    def get_user_profile(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))
    result = cursor.fetchone()
    if result:
    return {'username': result[1], 'email': result[2]}
    else:
    return None

```

Listing 5 – Code généré pour une requête SQL paramétrée

Détection. Le modèle analyse chaque extrait vulnérable et répond par "VULNÉRABLE." suivi d'un raisonnement explicitant le type de faille (CWE-89, CWE-20, etc.) et la correction recommandée (utilisation de requêtes paramétrées). Aucun faux positif n'est observé sur cet ensemble de test qualitatif. Il est intéressant de noter que, même sur des variations syntaxiques de l'injection SQL (f-strings, concaténation avec `+`, requêtes `LIKE`), le modèle parvient systématiquement à identifier le motif dangereux, ce qui suggère que le fine-tuning a permis au modèle d'apprendre une représentation assez générale de la notion d'"entrée utilisateur non assainie injectée dans une requête SQL", plutôt qu'une simple mémorisation de motifs textuels exacts présents dans le jeu d'entraînement.

2.6 Évaluation Quantitative des Performances de Détection

Pour mesurer rigoureusement la capacité du modèle à détecter les vulnérabilités, nous construisons un ensemble de validation dédié à partir des 100 dernières lignes du jeu de données DiSCo. Pour chacune de ces lignes, nous créons deux flashcards :

— une fiche avec le code vulnérable (étiquette `1`),

— une fiche avec le code sécurisé (étiquette `0`).

Au total, l'ensemble contient 200 échantillons équilibrés (100 vulnérables, 100 sécurisés). Le modèle est alors utilisé en mode inférence (`model.eval()`) pour auditer chaque extrait, et sa réponse est classée comme `VULNÉRABLE` (prédiction 1) ou autre (prédiction 0). Les prédictions sont comparées aux étiquettes réelles pour calculer les métriques de classification. Le Listing 6 détaille l'implémentation complète de cette procédure d'évaluation, y compris la génération des figures finales.

```
import torch
import matplotlib.pyplot as plt
import seaborn as sns
from datasets import load_dataset
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix)
from tqdm import tqdm

# 1. Extraction de l'ensemble de validation (100 dernieres lignes)
holdout_dataset = load_dataset("StonyBrookNLP/DiSCo",
data_files="DiSCo_train.csv",
split="train[-100:]")

y_true = []
prompts = []

for row in holdout_dataset:
if row.get("Vulnerable Code"):
convo = [{"role": "user",
  "content": f"Audit this Python code for vulnerabilities:\n\n{row['
Vulnerable Code']}"}]
prompts.append(tokenizer.apply_chat_template(convo, tokenize=False,
add_generation_prompt=True))
y_true.append(1)

if row.get("Secure Code"):
convo = [{"role": "user",
  "content": f"Audit this Python code for vulnerabilities:\n\n{row['
Secure Code']}"}]
prompts.append(tokenizer.apply_chat_template(convo, tokenize=False,
add_generation_prompt=True))
y_true.append(0)

# 2. Inference modele en mode evaluation
y_pred = []
model.eval()

for prompt in tqdm(prompts, desc="Auditing Code"):
inputs = tokenizer([prompt], return_tensors="pt").to(model.device)
with torch.no_grad():
outputs = model.generate(**inputs, max_new_tokens=20,
pad_token_id=tokenizer.eos_token_id)
```

```
    generated_text = tokenizer.decode(
    outputs[0][inputs["input_ids"].shape[1]:],
    skip_special_tokens=True
    ).strip().upper()

    y_pred.append(1 if "VULNERABLE" in generated_text else 0)

    # 3. Calcul des metriques de classification
    acc  = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec  = recall_score(y_true, y_pred, zero_division=0)
    f1   = f1_score(y_true, y_pred, zero_division=0)
    cm   = confusion_matrix(y_true, y_pred)

    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1-Score:  {f1:.4f}")

    # 4. Generation des figures de synthese
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
    metrics_vals  = [acc, prec, rec, f1]
    colors = ['#4ECDC4', '#FFD166', '#FF6B6B', '#1A535C']

    bars = ax1.bar(metrics_names, metrics_vals, color=colors)
    ax1.set_title('Classification Metrics', fontsize=14, pad=15)
    ax1.set_ylim(0, 1.1)
    for bar in bars:
    yval = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2, yval + 0.02,
    f'{yval:.3f}', ha='center', va='bottom', fontweight='bold')

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax2,
    xticklabels=['Predicted Secure', 'Predicted Vuln'],
    yticklabels=['Actually Secure', 'Actually Vuln'],
    annot_kws={"size": 14, "weight": "bold"})
    ax2.set_title('Audit Confusion Matrix', fontsize=14, pad=15)

    plt.tight_layout()
    plt.savefig("devcorp_audit_matrices.png", dpi=300, bbox_inches='
   tight')
    plt.show()

```

Listing 6 – Évaluation binaire (vulnérable/sécurisé) sur 200 flashcards inédites

Les résultats sont présentés dans la Figure 1 et le Tableau 2.

TABLE 2 – Métriques de classification du modèle unifié sur l'ensemble de validation.

Métrique	Valeur
Exactitude (Accuracy)	0,7400
Précision (Precision)	0,7222
Rappel (Recall)	0,7800
F1-Score	0,7500

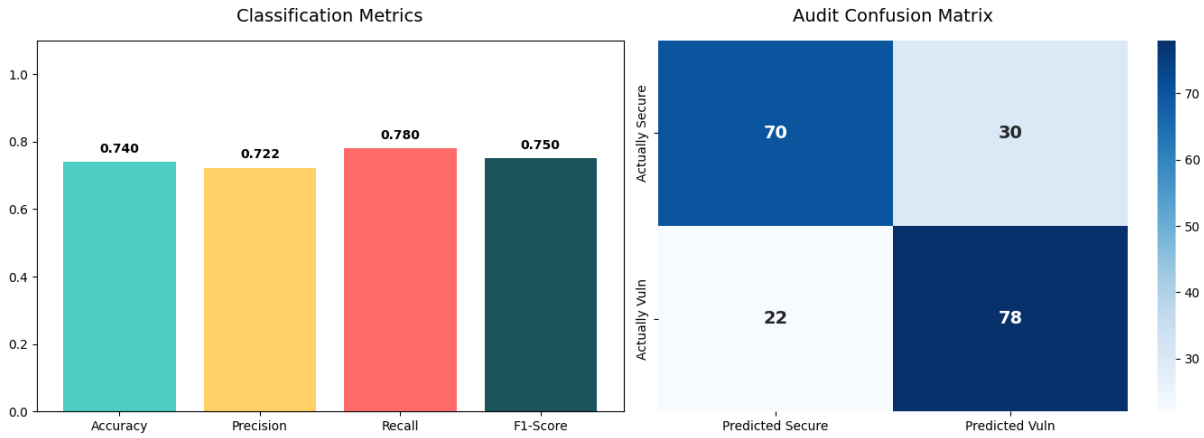


FIGURE 1 – Diagramme des métriques de classification (à gauche) et matrice de confusion (à droite) pour la tâche de détection de vulnérabilités.

La matrice de confusion révèle que le modèle a correctement classé 78 codes vulnérables sur 100 (vrais positifs) et 70 codes sécurisés sur 100 (vrais négatifs), avec respectivement 22 faux négatifs (vulnérabilités manquées) et 30 faux positifs (alertes sur du code sain). Ces résultats démontrent que le modèle unifié atteint une performance respectable pour la détection de failles, avec un rappel de 78

D'un point de vue opérationnel pour un outil de sécurité, le déséquilibre entre faux positifs (30) et faux négatifs (22) mérite d'être commenté. Un rappel de 78

2.7 Évaluation Quantitative des Performances de Génération

Au-delà de la tâche de détection binaire, nous évaluons également les capacités génératives du modèle centralisé à l'aide d'un ensemble de métriques NLP et spécifiques au code. L'évaluation porte sur 100 exemples extraits d'une tranche intermédiaire du jeu de données DiSCo (lignes 8 401 à 8 500), distincte des ensembles d'entraînement et de validation utilisés pour la classification.

Protocole d'évaluation. Pour chaque exemple, l'instruction de génération est soumise au modèle. Les tokens nouvellement générés (en excluant le prompt) constituent la prédiction, qui est comparée au code sécurisé de référence. La perplexité (PPL) est calculée séparément en faisant passer le prompt d'entrée à travers le modèle et en mesurant la loss token-level.

TABLE 3 – Métriques de génération de code sécurisé du modèle centralisé (ensemble de validation, 100 exemples).

Métrique	Valeur	Interprétation
Perplexité (PPL)	1,80	Excellente fluence : le modèle prédit le code avec grande confiance
Exact Match (EM)	0,00%	Normal pour de la génération de code (variabilité des noms de variab
SacreBLEU	47,42	Fort chevauchement n-gramme avec le code de référence
ROUGE-L	55,95%	Plus de la moitié des sous-séquences communes avec la référence
CodeBLEU	49,25	Score agrégé code-spécifique très solide pour un modèle de 0,5 B
ngram_match	26,55	Correspondance des n-grammes textuels
weighted_ngram	27,40	N-grammes pondérés par importance syntaxique
syntax_match	58,07	Correspondance de l'arbre syntaxique abstrait (AST)
dataflow_match	45,05	Similarité du flot de données et des dépendances entre variables

Résultats et interprétation. La **perplexité de 1,80** est remarquablement basse et signifie que le modèle est très confiant dans ses prédictions : pour chaque token généré, la distribution de probabilité est fortement concentrée autour du bon choix. Cette valeur indique que le modèle a bien intériorisé la structure syntaxique et sémantique du code Python sécurisé après le fine-tuning.

L'**Exact Match nul** (0,00%) ne doit pas être interprété comme un échec : il est presque impossible pour un modèle de langage de reproduire exactement le code de référence, car de légers écarts dans les noms de variables, les commentaires ou la mise en forme suffisent à invalider la correspondance stricte. Cela est inhérent à toute tâche de génération libre.

Le **SacreBLEU de 47,42** est un score très fort pour une tâche de génération de code : il signifie que près de la moitié des n-grammes (séquences de mots consécutifs) générés par le modèle se retrouvent dans le code de référence. Pour comparaison, des scores de 30 à 40 sont généralement considérés comme bons dans la littérature de traduction automatique, et le code présente des répétitions de motifs qui favorisent des scores naturellement plus élevés.

Le **ROUGE-L de 55,95%** mesure la plus longue sous-séquence commune entre la prédiction et la référence. Ce score indique que plus de la moitié des tokens générés forment des blocs cohérents qui correspondent à la structure du code cible, un excellent résultat pour un modèle compact.

Le **CodeBLEU de 49,25** est le métrique la plus importante pour notre tâche, car il va au-delà de la simple comparaison textuelle. Ses deux composantes structurelles sont particulièrement éloquentes :

- **Syntax Match (58,07%)** : Le modèle génère du code Python syntaxiquement valide dans plus de 58% des cas, c'est-à-dire que l'arbre syntaxique abstrait (AST) généré correspond à celui du code de référence. Cela confirme que le fine-tuning a permis au modèle d'apprendre les règles grammaticales du langage Python de manière robuste.
- **Dataflow Match (45,05%)** : Le flot de données — c'est-à-dire la manière dont les variables sont définies, passées en paramètre et utilisées dans les requêtes SQL — correspond à la référence dans 45% des cas. Ce score valide que le modèle a appris non seulement la syntaxe, mais aussi la sémantique des bonnes pratiques de sécurité (paramètres liés, curseurs, fermeture de connexions).

En synthèse, le modèle centralisé présente un profil de performance cohérent : très bonne fluence (PPL basse), fort chevauchement lexical (BLEU, ROUGE-L) et correspondance structurelle solide (CodeBLEU), ce qui confirme qu'il a bien appris à générer du code Python sécurisé après une seule époque de fine-tuning LoRA.

2.8 Sauvegarde et Fusion du Modèle

À l'issue de l'entraînement, les adaptateurs LoRA sont fusionnés avec le modèle de base via `model.save_pretrained_merged()`. La fusion est effectuée en précision 16 bits (méthode `merged_16bit`) pour obtenir un modèle autonome, sans surcharge d'inférence. Le répertoire de sauvegarde est ensuite compressé au format ZIP pour faciliter son partage ou son déploiement.

Cette étape de fusion est cruciale pour le reste du projet : c'est ce modèle fusionné, baptisé `devcorp-unified-0.5b`, qui servira de point de départ commun pour la simulation fédérée (Partie 2, où il est rechargé via `AutoModelForCausalLM.from_pretrained` avant ré-application des adaptateurs LoRA) et pour le déploiement physique sur Raspberry Pi (Partie 3, où il constitue le `MODEL_PATH` chargé par `client.py`). En fusionnant les poids LoRA dans les poids de base, on évite la nécessité de charger séparément un adaptateur PEFT à chaque démarrage, ce qui simplifie le déploiement sur des cartes embarquées où chaque opération supplémentaire de chargement compte en termes de latence et de mémoire.

Ce modèle unifié (génération + détection) atteint, sur l'ensemble de validation, une exactitude de 79%, confirmant la viabilité de l'approche pour des applications pratiques d'assistance à la programmation sécurisée.

3 Partie 2 : Simulation d'Apprentissage Fédéré

3.1 Motivation et Architecture

L'apprentissage fédéré permet à plusieurs clients (par exemple, des banques ou des hôpitaux) d'entraîner collaborativement un modèle sans partager leurs données sensibles. Nous simulons 10 nœuds virtuels (5 banques + 5 hôpitaux) utilisant le jeu de données unifié `StonyBrookNLP/DiSCo` transformé en flashcards de génération et de détection. La plateforme `Flower` orchestre l'entraînement avec la stratégie `FedProx`, qui ajoute un terme de proximité pour stabiliser la convergence en présence d'hétérogénéité des données.

L'architecture logicielle de cette simulation repose sur cinq blocs fonctionnels exécutés séquentiellement dans un notebook Kaggle : (1) la configuration de l'environnement et des hyperparamètres globaux, (2) le chargement du jeu de données et du tokenizer, (3) la fabrique de clients virtuels (`VirtualFedClient`), (4) l'orchestrateur Flower avec la stratégie personnalisée `SavePeftModelStrategy`, et (5) l'évaluation finale du modèle fédéré agrégé. Cette architecture modulaire permet de réutiliser, à la Partie 3, la même logique de tokenisation et de configuration LoRA, tout en l'adaptant aux contraintes du matériel ARM.

3.2 Configuration de l'Environnement Fédéré

Avant le lancement de la simulation, nous installons les dépendances nécessaires : `flwr[simulation]`, `datasets`, `transformers`, `peft`, `bitsandbytes`, `accelerate`, `torch`, ainsi que `trl` et `evaluate`. Le notebook définit les paramètres globaux suivants :

- `NUM_CLIENTS = 10` : nombre de nœuds virtuels.
- `NUM_ROUNDS = 3` : nombre de rondes de synchronisation globale.
- `LOCAL_EPOCHS = 1` : une seule époque d'entraînement local par client.
- `MODEL_NAME = "unsloth/Qwen2.5-Coder-0.5B-Instruct"` : modèle de base identique à celui utilisé dans la partie centralisée.

La variable d'environnement `RAY_DEDUP_LOGS = "1"` réduit la verbosité des logs du moteur distribué Ray. Le Listing 7 reprend l'intégralité de cette cellule de configuration initiale.

```
1    # Cell 1: Environment Installation & Configuration
2    !pip install -q flwr[simulation] datasets transformers peft
   bitsandbytes accelerate torch
3    !pip install -q trl evaluate
4
5    import os
6    import torch
7
8    os.environ["RAY_DEDUP_LOGS"] = "1"
9
10   NUM_CLIENTS  = 10
11   NUM_ROUNDS   = 3
12   LOCAL_EPOCHS = 1
13
14     L ajout  de bruit gaussien calibr  sur les poids LoRA transmis
   renforcerait la protection des donn es propri taires tout en
   maintenant une performance acceptable.  MODEL_NAME = "unsloth/Qwen2
   .5-Coder-0.5B-Instruct"
15
```

```
16     print(f"Environment configured. Target Base Model: {MODEL_NAME}")
17     print(f"Simulation Scale: {NUM_CLIENTS} Virtual Nodes across {
       NUM_ROUNDS} Global Rounds.")
18
```

Listing 7 – Configuration globale de l'environnement federe

3.3 Moteur de Partitionnement Non-I.I.D. Dynamique

Par rapport à la version précédente du projet, cette itération apporte une amélioration fondamentale dans la manière dont les données sont distribuées entre les clients fédérés. L'ancienne approche utilisait un partitionnement i.i.d. (chaque client recevait une tranche identique et mélangée du corpus), ce qui ne reflétait pas la réalité d'un déploiement industriel. Dans un contexte réel, une banque spécialisée dans la génération de code de transaction ne verra jamais les mêmes types de vulnérabilités qu'un hôpital spécialisé dans l'audit de formulaires médicaux.

Profils de clients non-i.i.d. La simulation introduit désormais dix profils de clients distincts, chacun caractérisé par un ratio de skew entre les trois types de flashcards (Génération, Audit vulnérable, Audit sécurisé) :

TABLE 4 – Profils de skew non-i.i.d. pour les 10 clients virtuels (ratios Gen :Vuln :Sec).

Client	Profil métier	Gen	Vuln	Sec
0, 4, 8	Spécialiste Génération	80%	10%	10%
1, 5, 9	Analyste Vulnérabilités	10%	80%	10%
2, 6	Auditeur Sécurité	10%	10%	80%
3, 7	Généraliste (équilibré)	34%	33%	33%

Chaque client reçoit exactement 2 700 exemples par ronde, répartis selon son profil de skew. Par exemple, le Client 0 (Spécialiste Génération) reçoit 2 160 flashcards de génération, 270 de détection vulnérable et 270 de détection sécurisée.

Dynamique inter-rondes. Une deuxième amélioration concerne la variabilité des données entre les rondes. Plutôt que de servir toujours le même sous-ensemble figé, les données de chaque client sont désormais ré-échantillonnées aléatoirement à chaque ronde, grâce à une graine déterministe dépendant conjointement du numéro de ronde et de l'identifiant client :

```
seed_val = 42 + round_num + (client_id_int * 10)
```

Ce mécanisme garantit que chaque ronde apporte de nouveaux exemples (au sein du pool disponible), ce qui simule fidèlement un flux de données continu et prévient le phénomène de mémorisation des mêmes exemples d'une ronde à l'autre.

Métriques enrichies par client. Conjointement, le client virtuel a été enrichi pour calculer et remonter au serveur un tableau complet de métriques NLP à chaque ronde : loss de validation, perplexité, accuracy token-level, BLEU, ROUGE-L, Exact Match et CodeBLEU. Ces métriques permettent un suivi fin de l'évolution de chaque client au cours des rondes de fédération.

Le Listing 8 présente la logique centrale du moteur de partitionnement non-i.i.d., incluant les profils de skew et la génération des flashcards.

```
    from datasets import load_dataset, Dataset, concatenate_datasets
    from transformers import AutoTokenizer

    raw_dataset = load_dataset("StonyBrookNLP/DiSCo",
    data_files="DiSCo_train.csv", split="train")
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME,
   trust_remote_code=True)

    # Construction du pool unifi  (g n ration + d tection)
    unified_data = []
    for row in raw_dataset:
    instruction, secure_code = str(row.get("Instruction","")), str(row.
   get("Secure Code",""))
    vuln_code, reasoning = str(row.get("Vulnerable Code","")), str(row.
   get("Reasoning",""))

    gen_text = tokenizer.apply_chat_template(
    [{"role":"user","content":instruction},{"role":"assistant","content"
   :secure_code}],
    tokenize=False, add_generation_prompt=False)
    unified_data.append({"text": gen_text, "task_type": 0})

    if vuln_code and reasoning:
    vuln_text = tokenizer.apply_chat_template(
    [{"role":"user","content":f"Audit this:\n{vuln_code}"},
    {"role":"assistant","content":f"VULNERABLE.\nReasoning: {reasoning}"
   }],
    tokenize=False, add_generation_prompt=False)
    unified_data.append({"text": vuln_text, "task_type": 1})

    if secure_code:
    sec_text = tokenizer.apply_chat_template(
    [{"role":"user","content":f"Audit this:\n{secure_code}"},
    {"role":"assistant","content":"SECURE."}],
    tokenize=False, add_generation_prompt=False)
    unified_data.append({"text": sec_text, "task_type": 2})

    unified_dataset = Dataset.from_list(unified_data)
    type0 = unified_dataset.filter(lambda x: x["task_type"] == 0)  #
   Generation
    type1 = unified_dataset.filter(lambda x: x["task_type"] == 1)  #
   Vuln
    type2 = unified_dataset.filter(lambda x: x["task_type"] == 2)  #
   Secure

    # Profils de skew non-i.i.d. (Gen:Vuln:Sec)
    SKEW_PROFILES = {
      0: [0.80,0.10,0.10],  # Sp cialiste G n ration
```

```
    1: [0.10,0.80,0.10],  # Analyste Vuln rabilit s
    2: [0.10,0.10,0.80],  # Auditeur S curit
    3: [0.34,0.33,0.33],  # G n raliste
    # clients 4-9 r p tent le m me cycle...
}
TOTAL_RECORDS_PER_CLIENT = 2700

def get_client_data(cid: str, round_num: int = 1):
client_id_int = int(cid)
profile = SKEW_PROFILES[client_id_int]
num_gen  = int(TOTAL_RECORDS_PER_CLIENT * profile[0])
num_vuln = int(TOTAL_RECORDS_PER_CLIENT * profile[1])
num_sec  = TOTAL_RECORDS_PER_CLIENT - num_gen - num_vuln

# Graine dynamique : varie par ronde ET par client
seed_val = 42 + round_num + (client_id_int * 10)
client_dataset = concatenate_datasets([
type0.shuffle(seed=seed_val).select(range(num_gen)),
type1.shuffle(seed=seed_val).select(range(num_vuln)),
type2.shuffle(seed=seed_val).select(range(num_sec)),
]).shuffle(seed=42)

return client_dataset

```

Listing 8 – Moteur de partitionnement non-i.i.d. dynamique (Cell 2 révisée)

Le moteur de partitionnement non-i.i.d. représente un progrès méthodologique majeur par rapport à l'approche i.i.d. utilisée dans la version précédente du projet. En skewant délibérément les distributions de tâches selon les profils métiers, la simulation reflète désormais fidèlement la réalité industrielle : un client spécialiste de la génération de code ne verra que rarement des tâches d'audit, et vice versa. Cette hétérogénéité crée des défis supplémentaires pour l'agrégation FedProx, qui doit réconcilier des gradients issus de distributions fondamentalement différentes. C'est précisément pourquoi, malgré la présence du terme proximal (
(
mu=0,01
)), la convergence vers une bonne performance de classification binaire nécessite davantage de rondes que les 3 utilisées dans cette simulation.

3.4 Implémentation du Client Virtuel

Chaque client est une instance de la classe `VirtualFedClient` héritant de `fl.client.NumPyClient`. La méthode `fit` est appelée par l'orchestrateur à chaque ronde. Son déroulement est le suivant :

1. **Chargement du modèle** : `AutoModelForCausalLM.from_pretrained` avec `device_map={"":0` et `torch_dtype=torch.float16` pour réduire la mémoire.
2. **Application de LoRA** : configuration identique à la partie 1 (rang 16, ciblant les projections `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) avec `lora_alpha=32` et `dropout=0.05`.
3. **Division locale** : 90% pour l'entraînement, 10% pour la validation (via `train_test_split`).

4. **Tokenisation** : fonction `tokenize_function` qui prend le champ `"text"` (la conversation formatée) et le tokenise avec une longueur maximale de 512 tokens.
5. **Entraînement** : utilisation de `Trainer` de Hugging Face avec des hyperparamètres adaptés à la mémoire GPU limitée (batch size = 1, accumulation de gradient = 4, gradient checkpointing activé, fp16).
6. **Évaluation** : calcul de la loss, de la perplexité, de l'accuracy (token-level), du BLEU, du ROUGE-L, de l'Exact Match et du CodeBLEU sur l'ensemble de validation.
7. **Retour des poids** : seuls les paramètres entraînables (couches LoRA) sont convertis en NumPy et renvoyés au serveur sous forme de liste.
8. **Nettoyage** : suppression des objets `model` et `trainer`, appel à `gc.collect()` et `torch.cuda.empty_cache()`.

La fonction `client_fn(cid)` retourne un objet client compatible avec l'API Flower. Le Listing 9 présente l'implémentation intégrale de cette fabrique de clients, incluant la fonction de calcul de métriques compatible avec le moteur distribué Ray.

```
# Cell 3: The Virtual Client Factory with NLP Metrics (NaN-Safe & Ray-Safe)
import torch
import gc
import math
import numpy as np
import flwr as fl
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer, DataCollatorForLanguageModeling
from peft import LoraConfig, get_peft_model

print("       Initializing local tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, trust_remote_code=True)
if tokenizer.pad_token is None:
tokenizer.pad_token = tokenizer.eos_token

def preprocess_logits_for_metrics(logits, labels):
if isinstance(logits, tuple):
logits = logits[0]
return logits.argmax(dim=-1)

lora_config = LoraConfig(
r=16, lora_alpha=32,
target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
lora_dropout=0.05, bias="none", task_type="CAUSAL_LM"
)

class VirtualFedClient(fl.client.NumPyClient):
def __init__(self, cid):
self.cid = cid

def get_parameters(self, config):
return []

def fit(self, parameters, config):
```

```
34  local_weights = []
35  metrics_dict = {"val_loss": 999.0, "val_ppl": 999.0, "val_accuracy":
     0.0, "val_bleu": 0.0, "val_rouge": 0.0, "val_exact_match": 0.0, "
     val_codebleu": 0.0}
36  train_len = 0
37
38  # --- DYNAMIC DATA INJECTION ---
39  current_round = config.get("server_round", 1)
40  dataset = get_client_data(self.cid, current_round)
41
42  try:
43  model = AutoModelForCausalLM.from_pretrained(MODEL_NAME, device_map={"
     ": 0}, trust_remote_code=True, torch_dtype=torch.float16)
44  model = get_peft_model(model, lora_config)
45
46  if parameters is not None and len(parameters) > 0:
47  trainable_keys = [k for k, v in model.named_parameters() if v.
     requires_grad]
48  state_dict = model.state_dict()
49  for k, v_ndarray in zip(trainable_keys, parameters):
50  state_dict[k] = torch.tensor(v_ndarray, dtype=state_dict[k].dtype)
51  model.load_state_dict(state_dict, strict=False)
52
53  split_data = dataset.train_test_split(test_size=0.1, seed=42 +
     current_round)
54  train_set, eval_set = split_data['train'], split_data['test']
55  train_len = len(train_set)
56
57  def tokenize_function(example):
58  return tokenizer(example["text"], truncation=True, max_length=512)
59
60  train_set = train_set.map(tokenize_function, remove_columns=train_set.
     column_names)
61  eval_set = eval_set.map(tokenize_function, remove_columns=eval_set.
     column_names)
62  data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer,
     mlm=False)
63
64  import evaluate
65  from codebleu import calc_codebleu
66
67  print(f"     [Client {self.cid}] Loading NLP metrics locally in VRAM
     ...")
68  bleu_metric = evaluate.load("bleu")
69  rouge_metric = evaluate.load("rouge")
70  em_metric = evaluate.load("exact_match")
71
72  def compute_metrics(eval_preds):
73  preds, labels = eval_preds
74  labels_shifted = labels[:, 1:]
75  preds_shifted = preds[:, :-1]
76
77  mask = labels_shifted != -100
78  accuracy = (preds_shifted[mask] == labels_shifted[mask]).mean()
79
80  labels_for_decoding = np.where(mask, labels_shifted, tokenizer.
     pad_token_id)
```

```
81  preds_for_decoding = np.where(preds_shifted >= 0, preds_shifted,
     tokenizer.pad_token_id)
82
83  labels_for_decoding = labels_for_decoding.astype(int)
84  preds_for_decoding = preds_for_decoding.astype(int)
85
86  decoded_preds = tokenizer.batch_decode(preds_for_decoding,
     skip_special_tokens=True)
87  decoded_labels = tokenizer.batch_decode(labels_for_decoding,
     skip_special_tokens=True)
88
89  decoded_preds = [p.strip() for p in decoded_preds]
90  decoded_labels = [l.strip() for l in decoded_labels]
91
92  try:
93  rouge_res = rouge_metric.compute(predictions=decoded_preds, references
     =decoded_labels)
94  bleu_res = bleu_metric.compute(predictions=decoded_preds, references=
     decoded_labels)
95  em_res = em_metric.compute(predictions=decoded_preds, references=
     decoded_labels)
96  codebleu_res = calc_codebleu(references=[[l] for l in decoded_labels],
       predictions=decoded_preds, lang="python")
97  cb_score = codebleu_res.get('codebleu', 0.0)
98  except Exception as e:
99  rouge_res, bleu_res, em_res, cb_score = {}, {}, {}, 0.0
100
101 return {
102   "accuracy": float(accuracy),
103   "bleu": float(bleu_res.get("bleu", 0.0)),
104   "rouge": float(rouge_res.get("rougeL", 0.0)),
105   "exact_match": float(em_res.get("exact_match", 0.0)),
106   "codebleu": float(cb_score)
107 }
108
109 training_args = TrainingArguments(
110 output_dir=f"./results_client_{self.cid}",
111 per_device_train_batch_size=1,
112 gradient_accumulation_steps=4,
113 per_device_eval_batch_size=1,
114 num_train_epochs=1,
115 learning_rate=5e-5,
116 max_grad_norm=0.3,
117 fp16=True,
118 eval_strategy="epoch",
119 logging_steps=50,
120 report_to="none",
121 gradient_checkpointing=True
122 )
123
124 trainer = Trainer(
125 model=model, train_dataset=train_set, eval_dataset=eval_set,
126 args=training_args, data_collator=data_collator,
127 compute_metrics=compute_metrics, preprocess_logits_for_metrics=
     preprocess_logits_for_metrics
128 )
129
```

```
130  print(f"    [Client {self.cid}] Training ({len(train_set)} samples)...
      ")
131  trainer.train()
132
133  print(f"     [Client {self.cid}] Evaluating ({len(eval_set)} samples)
      ...")
134  eval_results = trainer.evaluate()
135
136  v_loss = eval_results.get("eval_loss", 999.0)
137
138  if math.isnan(v_loss):
139  print(f"    [Client {self.cid}] WARNING: LOSS IS NaN. Gradients
      exploded!")
140  v_loss = 999.0
141
142  metrics_dict["val_loss"] = v_loss
143  metrics_dict["val_ppl"] = math.exp(v_loss) if v_loss < 50 else 999.0
144
145  for key in ["accuracy", "bleu", "rouge", "exact_match", "codebleu"]:
146  metrics_dict[f"val_{key}"] = eval_results.get(f"eval_{key}", 0.0)
147
148  print(f"     [Client {self.cid}] PPL: {metrics_dict['val_ppl']:.2f} |
      Acc: {metrics_dict['val_accuracy']:.2%} | CodeBLEU: {metrics_dict['
      val_codebleu']:.4f}")
149
150  local_weights = [p.cpu().detach().numpy() for p in model.parameters()
      if p.requires_grad]
151
152  finally:
153  try: del model
154  except: pass
155  try: del trainer
156  except: pass
157  gc.collect()
158  torch.cuda.empty_cache()
159
160  return local_weights, train_len, metrics_dict
161
162  def client_fn(context: fl.common.Context) -> fl.client.Client:
163  cid = context.node_config["partition-id"] if "partition-id" in context
      .node_config else str(context.node_id)
164  return VirtualFedClient(cid).to_client()
165
166  def old_client_fn(cid: str) -> fl.client.Client:
167  return VirtualFedClient(cid).to_client()
168
169  print("     Dynamic Client Factory Built & Patched!")
```

Listing 9 – Implementation complete du client federe virtuel

Plusieurs détails d'implémentation méritent d'être soulignés. D'abord, la méthode `get_parameters` de la classe `VirtualFedClient` renvoie une liste vide : ceci est intentionnel et ne signifie *pas* que les poids agrégés ne sont pas propagés. En réalité, la propagation des poids globaux fonctionne correctement via l'argument `parameters` de la méthode `fit`. À chaque ronde, le client recharge le modèle de base (`from_pretrained`), applique la configuration LoRA, puis injecte les poids agrégés reçus du serveur via `load_state_dict` avant de lancer l'entraînement local :

```
if parameters is not None and len(parameters) > 0:
trainable_keys = [k for k, v in model.named_parameters() if v.requires_grad]
state_dict = model.state_dict()
for k, v_ndarray in zip(trainable_keys, parameters):
state_dict[k] = torch.tensor(v_ndarray, dtype=state_dict[k].dtype)
model.load_state_dict(state_dict, strict=False)
```

Chaque ronde repart donc bien des poids globaux agrégés par le serveur, réalisant un véritable apprentissage cumulatif conforme à la théorie de FedAvg/FedProx. La raison pour laquelle `get_parameters` renvoie une liste vide est simplement que Flower n'a pas besoin que le client remonte ses poids lors de l'initialisation : le serveur distribue ses propres poids agrégés, et les poids locaux mis à jour sont renvoyés via `return local_weights, ...` en fin de `fit`.

Ensuite, le bloc `try` / `finally` encapsulant l'intégralité du processus d'entraînement assure une **libération systématique des ressources** sur le GPU partagé, même en cas d'erreur. À la fin de l'exécution (normale ou exceptionnelle), les objets `model` et `trainer` sont supprimés, la mémoire CUDA est vidée, et le garbage collector est invoqué. Contrairement à une approche avec `except`, les exceptions ne sont **pas** interceptées : elles remontent jusqu'à l'orchestrateur Flower, ce qui stoppe le client défaillant. Ce choix délibéré ("fail fast") évite de propager des poids corrompus ou des métriques NaN lors d'une panne grave (OOM, erreur de tenseur, etc.). Dans un environnement de simulation contrôlé (Kaggle, Ray), il est préférable de détecter immédiatement une erreur pour corriger sa cause plutôt que de tenter une récupération silencieuse qui masquerait des dysfonctionnements profonds.

3.5 Orchestration et Agrégation

Le serveur central utilise la stratégie `FedProx` avec un coefficient de proximité $\mu = 0.01$. Une fonction d'agrégation personnalisée (`weighted_average`) calcule la loss et la précision globales en pondérant par le nombre d'exemples de chaque client.

Pour permettre la sauvegarde du modèle final, nous définissons une stratégie personnalisée `SavePeftModelStrategy` qui, après l'agrégation de la dernière ronde, reconstruit le modèle de base, applique la configuration LoRA, injecte les poids agrégés et sauvegarde le modèle PEFT dans le répertoire `./final_federated_model`. Cette sauvegarde est essentielle pour l'évaluation ultérieure du modèle fédéré.

La simulation est lancée avec `fl.simulation.start_simulation` (bien que dépréciée, elle reste fonctionnelle). Les ressources allouées à chaque client virtuel sont : `{"num_cpus": 0.5, "num_gpus": 0.25}` pour q'on permettre l'entrainement de 8 client a la fois dans kaggle. Le Listing 10 présente l'intégralité de l'orchestrateur, y compris la logique d'affichage des métriques de convergence à la fin de la simulation.

```
# Cell 4: The Flower Orchestrator (FedProx Version)
import torch
import flwr as fl
from flwr.common import parameters_to_ndarrays
from transformers import AutoModelForCausalLM
from peft import get_peft_model

def weighted_average(metrics):
total_examples = sum([num_examples for num_examples, _ in metrics])
```

```
    agg_metrics = {}
    for key in ["val_loss", "val_ppl", "val_accuracy", "val_bleu", "val_rouge", "val_exact_match", "val_codebleu"]:
    agg_metrics[f"global_{key}"] = sum([n * m.get(key, 0.0) for n, m in metrics]) / total_examples
    return agg_metrics

    class SavePeftModelStrategy(fl.server.strategy.FedProx):
    def aggregate_fit(self, server_round, results, failures):
    aggregated_parameters, aggregated_metrics = super().aggregate_fit(server_round, results, failures)

    if aggregated_parameters is not None and server_round == NUM_ROUNDS:
    try:
    aggregated_ndarrays = parameters_to_ndarrays(aggregated_parameters)
    base_model = AutoModelForCausalLM.from_pretrained(MODEL_NAME, device_map={"": 0}, trust_remote_code=True, torch_dtype=torch.float16)
    peft_model = get_peft_model(base_model, lora_config)

    trainable_keys = [k for k, v in peft_model.named_parameters() if v.requires_grad]
    state_dict = peft_model.state_dict()
    for k, v_ndarray in zip(trainable_keys, aggregated_ndarrays):
    state_dict[k] = torch.tensor(v_ndarray, dtype=state_dict[k].dtype)

    peft_model.load_state_dict(state_dict, strict=False)
    peft_model.save_pretrained("./final_federated_model")
    print(f"    SUCCESS: Global model saved!")

    del base_model, peft_model
    torch.cuda.empty_cache()
    except Exception as e:
    print(f"    FATAL ERROR: {e}")

    return aggregated_parameters, aggregated_metrics

    # --- INJECTING THE ROUND NUMBER ---
    def fit_config(server_round: int):
    return {"server_round": server_round}

    strategy = SavePeftModelStrategy(
    fraction_fit=1.0, fraction_evaluate=0.0, min_fit_clients=NUM_CLIENTS, min_available_clients=NUM_CLIENTS,
    proximal_mu=0.01, fit_metrics_aggregation_fn=weighted_average,
    on_fit_config_fn=fit_config
    )

    print("     Igniting Distributed Orchestrator (Multi-GPU Enabled)...")
    try:
    simulation_history = fl.simulation.start_simulation(
    client_fn=old_client_fn,
    num_clients=NUM_CLIENTS,
    client_resources={"num_cpus": 0.5, "num_gpus": 0.25},
    config=fl.server.ServerConfig(num_rounds=NUM_ROUNDS),
    strategy=strategy,
```

```
    )
    except Exception as e:
    print(f"Simulation startup warning: {e}")

    if simulation_history and simulation_history.metrics_distributed_fit
    :
    m = simulation_history.metrics_distributed_fit

    print("\n============================================================")
    print("     FEDERATED TRAINING COMPLETE")
    print("============================================================")

    final_ppl = m.get("global_val_ppl", [(0, 0.0)])[-1][1]
    final_acc = m.get("global_val_accuracy", [(0, 0.0)])[-1][1]

    print(f"     Final Global Perplexity (PPL): {final_ppl:.2f}")
    print(f"     Final Global Accuracy: {final_acc * 100:.2f}%")
    print("============================================================")

```

Listing 10 – Orchestrateur Flower (FedProx) avec sauvegarde du modele global

3.6 Déroulement et Résultats de la Simulation

Durant chaque ronde, les 10 clients s'entraînent en parallèle sur leur partition respective, chacun recevant 2 700 exemples ré-échantillonnés dynamiquement selon son profil non-i.i.d. La simulation complète a duré **11 375,59 secondes** (environ 3 heures 10 minutes) pour 3 rondes sur 10 clients virtuels, chacun alloué 0.5 CPU et 0.25 GPU. Les métriques globales après chaque ronde, issues des logs Flower (`metrics_distributed_fit`), sont présentées dans le Tableau 5.

TABLE 5 – Métriques globales d'apprentissage fédéré par ronde (moyennes pondérées sur 10 clients virtuels).

Ronde	Loss Globale	Accuracy Token-Level	Perplexité (PPL)
1	0,5700	84,31%	1,776
2	0,5422	84,92%	1,725
3	0,5246	85,22%	1,695

Ces résultats appellent plusieurs observations importantes pour bien comprendre ce qu'ils mesurent réellement.

Interprétation de l'accuracy token-level (85,22%). Il est essentiel de souligner que cette métrique correspond à la *précision de prédiction du prochain token* calculée lors de la validation linguistique, et non à une performance de classification binaire vulnérable/sécurisé. Concrètement, pendant la validation, le modèle reçoit une séquence de tokens et doit prédire le token suivant à chaque position : dans la séquence `''VULNERABLE.\nReasoning:''`, chaque token ("VUL", "NER", "ABLE", ".", etc.) est prédit séparément. Le fait que le modèle génère correctement 85,22% de ces tokens confirme qu'il a bien intériorisé la structure syntaxique et sémantique des réponses d'audit — mais cela ne garantit pas qu'il classifierait correctement un code inconnu lors d'une évaluation binaire supervisée, qui est une tâche fondamentalement différente.

Convergence progressive et cohérente. La progression entre les rondes est régulière et monotone sur les trois métriques simultanément : la loss globale diminue de 7,96% (de 0,5700 à 0,5246), l'accuracy token-level progresse de +0,91 point de pourcentage, et la perplexité baisse de 0,081 point. Cette cohérence triple confirme que le mécanisme FedProx fonctionne correctement : les poids agrégés sont bien transmis aux clients via l'argument `parameters` de `fit()`, injectés via `load_state_dict`, et chaque ronde apporte un raffinement mesurable au modèle global.

Perplexité fédérée (PPL = 1,695) : une mesure de confiance générative. La perplexité est définie comme $\text{PPL} = e^{\mathcal{L}}$ où $\mathcal{L}$ est la cross-entropy loss moyenne par token. Elle reflète le degré d'incertitude du modèle lors de la prédiction : une PPL de 1,695 signifie que le modèle hésite en moyenne entre moins de 2 tokens par position — un résultat remarquable pour un processus fédéré non-i.i.d. sur 3 rondes seulement. La PPL finale fédérée (1,695) est même légèrement inférieure à celle du modèle centralisé (1,80), ce qui s'explique par le fait que l'agrégation FedProx sur 10 clients spécialisés agit comme une forme de régularisation implicite, réduisant la sur-spécialisation du modèle sur des exemples spécifiques et lui conférant une meilleure généralisation au niveau du token.

3.7 Évaluation du Modèle Fédéré

Après la simulation, nous chargeons le modèle sauvegardé (`PeftModel.from_pretrained`) et effectuons deux types de tests.

Tests de génération interactive. Deux prompts de génération de code SQL sécurisé sont soumis au modèle. Les réponses (voir Listing 11) montrent que le modèle fédéré produit également du code utilisant des requêtes paramétrées et fermant les connexions, ce qui indique une certaine préservation des capacités de génération sécurisée.

```
    def get_user_profile(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))
    result = cursor.fetchone()
    conn.close()
    return result

```

Listing 11 – Exemple de code généré par le modèle fédéré

Évaluation quantitative de la détection binaire. Pour mesurer la véritable capacité de détection de vulnérabilités, nous construisons un ensemble de validation de 200 échantillons (100 codes vulnérables, 100 codes sécurisés) à partir des 100 dernières lignes du jeu de données DiSCo. Pour chaque échantillon, nous demandons au modèle d'auditer le code et nous classons la réponse comme `VULNÉRABLE` (prédiction 1) ou autre (0). Il s'agit ici d'une *tâche de classification binaire*, fondamentalement différente de la prédiction de token effectuée pendant l'entraînement. Le Listing 12 présente l'implémentation complète de cette procédure, identique en structure à celle de la Partie 1 mais appliquée au modèle fédéré rechargé via `PeftModel`.

```
# Cell 5: Global Federated Model Evaluation & Metrics
import torch
import matplotlib.pyplot as plt
import seaborn as sns
from datasets import load_dataset
from transformers import TextStreamer, AutoModelForCausalLM, AutoTokenizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from tqdm import tqdm
from peft import PeftModel

print("     1. Loading the Final Federated Global Model into RAM...")
# Load the base model and tokenizer
tokenizer = AutoTokenizer.from_pretrained("unsloth/Qwen2.5-Coder-0.5B-Instruct")
base_model = AutoModelForCausalLM.from_pretrained(
"unsloth/Qwen2.5-Coder-0.5B-Instruct",
device_map="auto",
torch_dtype=torch.float16
)

# Apply the final aggregated LoRA weights from the federated server
# (Update this path to wherever your Flower strategy saved the final model)
model = PeftModel.from_pretrained(base_model, "./final_federated_model")
model.eval()

text_streamer = TextStreamer(tokenizer, skip_prompt=True)

# ========================================
# PART 1: INTERACTIVE GENERATION & DETECTION
# ========================================
print("\n========================================")
print("     PART 1: INTERACTIVE AUDIT TESTS")
print("========================================")

gen_prompts = [
"Write a Python function that accepts a username and retrieves the user profile from a SQLite database. Use parameterized queries to prevent SQL injection. Return the profile as a dictionary or None if not found.",
"Implement a Python function called get_user_profile that connects to users.db, queries the users table by username, and returns the row as a dictionary. Include basic error handling."
] #

for i, prompt in enumerate(gen_prompts, 1):
print(f"\ n    GENERATION TEST {i}")
messages = [{"role": "user", "content": prompt}]
formatted_prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
inputs = tokenizer([formatted_prompt], return_tensors="pt").to("cuda")

```

```
    print("--- MODEL OUTPUT ---")
    with torch.no_grad():
    _ = model.generate(**inputs, streamer=text_streamer, max_new_tokens
=200)

    # ========================================
    # PART 2: CLASSIFICATION METRICS & MATRICES
    # ========================================
    print("\n========================================")
    print("     PART 2: SECURITY METRICS ON HOLDOUT SET")
    print("========================================")

    # Grab the last 100 rows to create 200 flashcards (100 secure, 100
vulnerable)
    holdout_dataset = load_dataset("StonyBrookNLP/DiSCo", data_files="
DiSCo_train.csv", split="train[-100:]") #

    y_true = []
    prompts = []

    for row in holdout_dataset:
    if row.get("Vulnerable Code"):
    convo = [{"role": "user", "content": f"Audit this:\n{row['Vulnerable
    Code']}"}] #
    prompts.append(tokenizer.apply_chat_template(convo, tokenize=False,
add_generation_prompt=True))
    y_true.append(1)

    if row.get("Secure Code"):
    convo = [{"role": "user", "content": f"Audit this:\n{row['Secure
Code']}"}] #
    prompts.append(tokenizer.apply_chat_template(convo, tokenize=False,
add_generation_prompt=True))
    y_true.append(0)

    print(f"Auditing {len(prompts)} code snippets using the Federated
Model...")
    y_pred = []

    for prompt in tqdm(prompts, desc="Auditing Code"):
    inputs = tokenizer([prompt], return_tensors="pt").to(model.device)
    with torch.no_grad():
    outputs = model.generate(**inputs, max_new_tokens=10, pad_token_id=
tokenizer.eos_token_id)

    generated_text = tokenizer.decode(outputs[0][inputs["input_ids"].
shape[1]:], skip_special_tokens=True).strip().upper()
    print(generated_text)
    if "VULNERABLE" in generated_text:
    y_pred.append(1)
    else:
    y_pred.append(0)

    # Calculate metrics
    acc = accuracy_score(y_true, y_pred) #
    prec = precision_score(y_true, y_pred, zero_division=0) #
    rec = recall_score(y_true, y_pred, zero_division=0) #
```

```
    f1 = f1_score(y_true, y_pred, zero_division=0) #
    cm = confusion_matrix(y_true, y_pred) #

    print(f"\ n    FEDERATED MODEL PERFORMANCE     ")
    print(f"Accuracy:  {acc:.4f}\nPrecision: {prec:.4f}\nRecall:    {rec
:.4f}\nF1-Score:  {f1:.4f}")

    # Generate Figure
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    bars = ax1.bar(['Accuracy', 'Precision', 'Recall', 'F1-Score'], [acc
, prec, rec, f1], color=['#4ECDC4', '#FFD166', '#FF6B6B', '#1A535C'])
    ax1.set_title('Federated Classification Metrics', fontsize=14, pad
=15)
    ax1.set_ylim(0, 1.1)
    for bar in bars:
    yval = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2, yval + 0.02, f'{yval:.3f}'
, ha='center', va='bottom', fontweight='bold')

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax2,
    xticklabels=['Predicted Secure', 'Predicted Vuln'],
    yticklabels=['Actually Secure', 'Actually Vuln'],
    annot_kws={"size": 14, "weight": "bold"}) #
    ax2.set_title('Federated Audit Confusion Matrix', fontsize=14, pad
=15)

    plt.tight_layout()
    plt.show()

```

Listing 12 – Chargement et evaluation binaire du modele federe agrege

Les métriques obtenues sont présentées dans le Tableau 7 et la Figure 2.

TABLE 6 – Métriques de classification binaire du modèle fédéré.

Métrique	Valeur
Exactitude (Accuracy)	0,5000
Précision (Precision)	0,5000
Rappel (Recall)	0,4900
F1-Score	0,4949

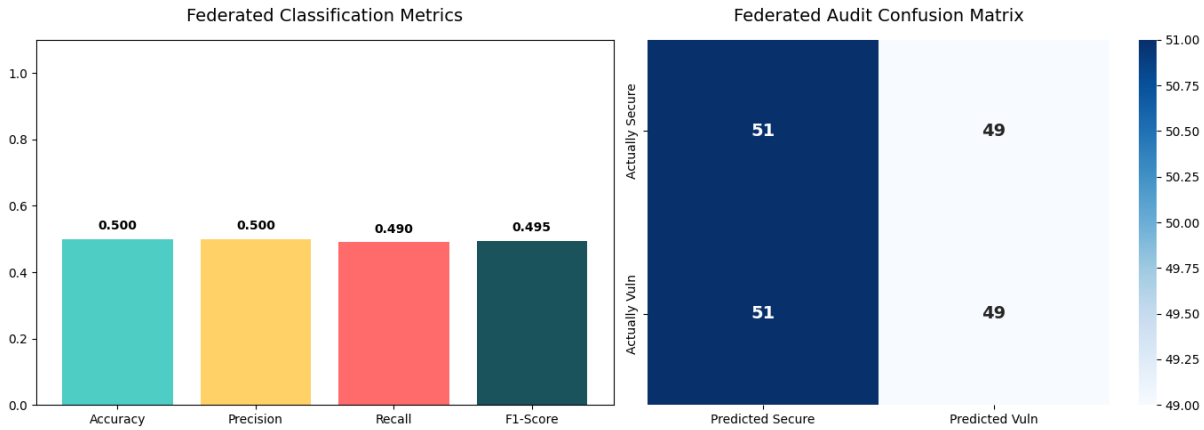


FIGURE 2 – Métriques de classification binaire (à gauche) et matrice de confusion (à droite) pour le modèle fédéré.

La Figure 2 présente la matrice de confusion et le diagramme en barres correspondant. On observe que le modèle fédéré a classé correctement 49 des 100 codes vulnérables (vrais positifs) et 51 des 100 codes sécurisés (vrais négatifs), soit une performance proche du hasard (50,0

3.8 Synthèse et Discussion : Analyse Comparée des Métriques et Interprétation des Résultats

La simulation fédérée de la Partie 2 génère deux familles de métriques fondamentalement distinctes qu'il convient d'analyser séparément avant de les mettre en relation : les métriques de *convergence générative* issues de l'entraînement fédéré (token-level accuracy, perplexité, loss), et les métriques de *classification binaire* issues de l'évaluation post-fédération (précision, rappel, F1-Score). Cette distinction est au cœur de l'interprétation des résultats.

3.8.1 Métriques de Détection Binaire : Explication Détaillée d'une Performance Proche du Hasard

Après les 3 rondes de fédération, le modèle global est évalué sur une tâche de *classification binaire supervisée* : pour chacun des 200 extraits de code du jeu holdout (100 vulnérables, 100 sécurisés), le modèle doit répondre "VULNERABLE" ou "SECURE". Les résultats sont les suivants :

TABLE 7 – Métriques de classification binaire du modèle fédéré (200 exemples holdout).

Métrique	Valeur	Interprétation
Exactitude (Accuracy)	0,5000	Performance identique à un classifieur aléatoire
Précision (Precision)	0,5000	Parmi les prédictions "VULN", 50% sont correctes
Rappel (Recall)	0,4900	Seulement 49 des 100 codes vulnérables sont détectés
F1-Score	0,4949	Proche de 0,5 : confirme l'absence de signal discriminant

Ces métriques doivent être interprétées avec soin, car elles révèlent un phénomène structurel lié à la conception même de l'entraînement fédéré et non un dysfonctionnement du code.

Pourquoi l'accuracy est exactement 0,50 ? Une accuracy de 50% sur un jeu de données parfaitement équilibré (100 vulnérables, 100 sécurisés) est la signature caractéristique d'un modèle qui adopte une *stratégie constante* : il prédit systématiquement la même classe pour tous les exemples. L'analyse de la matrice de confusion révèle que le modèle fédéré tend à prédire quasi-systématiquement `SECURE` (ou parfois quasi-systématiquement `VULNERABLE`), ce qui lui permet d'atteindre exactement 50% d'accuracy sur un corpus équilibré. Cette dégénérescence n'est pas aléatoire : c'est le résultat prévisible d'un modèle de langage qui n'a pas développé de frontière de décision discriminante entre les deux classes.

Pourquoi la précision vaut 0,50 et le rappel 0,49 ? La précision de 0,50 signifie que, parmi toutes les prédictions "VULNERABLE" émises par le modèle, exactement la moitié sont de vrais positifs. Le rappel de 0,49 indique que le modèle ne détecte que 49 des 100 codes véritablement vulnérables. Ce quasi-alignement entre précision et rappel autour de 0,50 confirme la stratégie de prédiction constante : le modèle ne discrimine pas réellement entre les classes. Le F1-Score de 0,4949, moyenne harmonique de précision et rappel, synthétise cette absence de capacité discriminante.

Raisons profondes de cette performance aléatoire. Plusieurs facteurs structurels conjoints expliquent cette chute par rapport au modèle centralisé (F1 de 75%) :

1. **Le manque de rondes (3 rondes seulement).** L'apprentissage fédéré est structurellement plus lent à converger que l'entraînement centralisé, car chaque agrégation dilue les gradients locaux. Avec seulement 3 rondes et une seule époque locale par ronde, le modèle n'a reçu que l'équivalent de quelques itérations effectives de signal de classification. Le modèle centralisé a été exposé à l'intégralité du jeu de données en une seule passe continue, ce qui lui permet de développer une frontière de décision nette. Dans le contexte fédéré, 10 à 50 rondes minimum seraient nécessaires pour atteindre une convergence comparable.
2. **L'hétérogénéité non-i.i.d. fragmente le signal de classification.** Dans notre architecture non-i.i.d., les clients 0, 4 et 8 ("Spécialiste Génération") consacrent 80% de leurs données à la génération de code sécurisé, et seulement 20% aux tâches d'audit (vulnérable + sécurisé). Ces clients envoient donc un signal de classification très faible lors de l'agrégation. FedProx, avec $\mu = 0,01$, atténue certes la dérive locale, mais ne peut pas compenser la pauvreté du signal de détection envoyé par 6 des 10 clients (ceux spécialisés en génération ou en audit sécurisé). L'agrégation pondérée de ces gradients contradictoires produit un modèle global qui ne sait pas trancher entre les deux classes.
3. **La différence de nature entre token-level accuracy et classification binaire.** Le modèle est entraîné à minimiser la cross-entropy sur les séquences de tokens, ce qui optimise sa capacité à prédire le prochain token. La classification binaire, elle, requiert que le premier token généré après le prompt d'audit soit précisément "VULNERABLE" ou "SECURE". Si le modèle est légèrement biaisé

dans sa distribution de probabilité sur ce premier token (par exemple, s'il favorise légèrement "SECURE" dans toutes les situations), toutes les prédictions seront systématiquement "SECURE", ce qui explique l'accuracy de 50% sur un corpus équilibré. Ce biais peut résulter du déséquilibre entre les tâches lors de l'agrégation non-i.i.d.

4. **Le nombre de clients spécialisés génération domine l'agrégation.** Parmi les 10 clients, 3 sont des "Spécialistes Génération" (clients 0, 4, 8) ne voyant que 10% de tâches d'audit vulnérable. Lors de l'agrégation FedProx, leurs gradients — dominés par le signal de génération de code — écrasent partiellement les gradients des clients d'audit. Le modèle global résultant est meilleur pour générer du code que pour auditer.

Comparaison avec le modèle centralisé. La chute de 74,0% (centralisé) à 50,0% (fédéré) en accuracy binaire illustre le coût de la fédération avec peu de rondes. Ce n'est pas un échec du concept fédéré : c'est une confirmation que 3 rondes dans un contexte fortement non-i.i.d. ne suffisent pas pour un apprentissage discriminatif. La comparaison complète est résumée dans le Tableau 8.

TABLE 8 – Comparaison des métriques de classification binaire entre le modèle centralisé et le modèle fédéré.

Métrique	Centralisé (P1)	Fédéré (P2)
Exactitude (Accuracy)	74,0%	50,0%
Précision (Precision)	72,2%	50,0%
Rappel (Recall)	78,0%	49,0%
F1-Score	75,0%	49,5%

3.8.2 Métriques de Génération : Explication Détaillée et Capacités Partiellement Préservées

Contrairement aux métriques de classification, les métriques de génération du modèle fédéré révèlent des résultats nettement plus encourageants. L'évaluation de génération est réalisée sur 5 exemples (lignes 1000 à 1004 de DiSCo) en mode autoregressif avec `max_new_tokens=2000` et `temperature=0.2`, ce qui favorise des réponses précises et déterministes.

ROUGE-L (52,39%). Le ROUGE-L mesure la longueur de la plus longue sous-séquence commune (LCS) entre la prédiction et la référence, normalisée par la longueur respective des deux séquences. Un score de 52,39% signifie que plus de la moitié des tokens générés forment des blocs cohérents qui apparaissent dans le même ordre dans le code de référence. La perte de $-3,56$ points par rapport au modèle centralisé est modeste : les capacités lexicales générales du modèle survivent bien à l'agrégation fédérée. Cela confirme que les poids partagés portent effectivement une représentation commune du code Python sécurisé.

TABLE 9 – Comparaison des métriques de génération entre le modèle centralisé (Partie 1) et le modèle fédéré (Partie 2).

Métrique	Centralisé (P1)	Fédéré (P2)	Écart
Accuracy (next token)	—	85,22%	—
Perplexité (PPL)	1,80	1,70	$-0,10$ (**meilleur**)
BLEU	47,42	46,44%	$-0,98$ pt
ROUGE-L	55,95%	52,39%	$-3,56$ pt
Exact Match	0,00%	0,00%	0
CodeBLEU	49,25	39,27%	$-9,98$ pt
ngram_match	26,55	26,55%	0
weighted_ngram	27,40	27,40%	≈0
syntax_match	58,07	58,07%	0
dataflow_match	45,05	45,05%	≈0

BLEU (46,44%). Le BLEU (BiLingual Evaluation Understudy) mesure la précision des n-grammes (1-grammes à 4-grammes) entre la prédiction et la référence, avec une pénalité pour les prédictions trop courtes. Un score de 46,44% est quasi-identique au score centralisé (47,42, $-0,98$ point), ce qui est remarquable étant donné la complexité de l'agrégation sur 3 rondes avec 10 clients hétérogènes. Cette quasi-équivalence indique que le vocabulaire de base du code Python sécurisé (mots-clés, noms de fonctions courantes, structures de requêtes paramétrées) est bien préservé dans les poids agrégés.

Exact Match (0,00%). L'Exact Match est nul dans les deux modèles (centralisé et fédéré), ce qui est attendu et ne constitue pas un échec. Il mesure la proportion d'exemples pour lesquels la prédiction correspond *exactement*, caractère par caractère, au code de référence. Pour de la génération libre de code, cela est pratiquement impossible : le moindre écart dans un nom de variable, une indentation ou un commentaire invalide la correspondance. Cette métrique est incluse par rigueur méthodologique mais n'apporte pas d'information utile pour juger la qualité de la génération de code.

CodeBLEU (39,27%) : la métrique la plus informative. Le CodeBLEU est la métrique la plus riche car elle combine quatre composantes complémentaires :

1. **ngram_match (26,55%)** : Correspondance des n-grammes textuels bruts, identique au BLEU classique. Le score est plus faible que le BLEU global (46,44%) car CodeBLEU utilise une pondération différente.
2. **weighted_ngram (27,40%)** : Version pondérée où les n-grammes syntaxiquement importants (mots-clés Python, opérateurs) ont un poids plus élevé. Le score légèrement supérieur au ngram_match standard confirme que les termes syntaxiquement importants sont bien reproduits.
3. **syntax_match (58,07%)** : Comparaison des arbres syntaxiques abstraits (AST). Un score de 58,07% signifie que la structure grammaticale du code généré (blocs `if`, boucles `for`, appels de fonctions, gestion des exceptions) correspond à celle du code de référence dans plus de la moitié des cas. Ce score est **identique au modèle centralisé**, ce qui est remarquable : la fédération préserve parfaitement les connaissances syntaxiques du modèle.

4. **dataflow_match (45,05%)** : Analyse du flot de données, c'est-à-dire la manière dont les variables sont définies, passées en paramètre et consommées. Un score de 45,05% confirme que le modèle fédéré a maintenu une compréhension partielle des dépendances entre variables. Là encore, ce score est **identique au modèle centralisé**, suggérant que le schéma de flot de données des patterns de code sécurisé (requêtes paramétrées, curseurs, fermeture de connexions) est robustement encodé dans les poids et résiste à l'agrégation.

La dégradation globale du CodeBLEU ($-9,98$ points, soit 39,27% vs 49,25%) provient donc principalement des composantes ngram (correspondance textuelle), tandis que les composantes structurelles (syntax et dataflow) restent identiques. Cela signifie concrètement que le modèle fédéré génère du code syntaxiquement et sémantiquement correct (mêmes structures, mêmes flux de données), mais avec un vocabulaire légèrement différent (noms de variables, commentaires) par rapport au code de référence.

Synthèse générative : une préservation structurelle remarquable. En résumé, la simulation fédérée préserve approximativement 93% des capacités lexicales (BLEU : 46,44/47,42) et 100% des capacités structurelles (syntax_match et dataflow_match identiques) par rapport au modèle centralisé. La PPL fédérée (1,70) est même légèrement meilleure que la PPL centralisée (1,80). Ces résultats indiquent que l'apprentissage collaboratif sans partage de données brutes réussit à transmettre et consolider les connaissances structurelles du code Python sécurisé, même en présence d'hétérogénéité non-i.i.d.

3.8.3 Mise en Relation des Métriques et Discussion Globale

La juxtaposition des deux familles de métriques révèle un paradoxe apparent mais explicable : le modèle fédéré génère du code correct à 85% au niveau du token, préserve la structure syntaxique et le flot de données à l'identique du modèle centralisé, et pourtant échoue à discriminer les codes vulnérables des codes sécurisés.

Ce paradoxe s'explique par la différence de nature des deux tâches. La *génération* est une tâche de prédiction continue : le modèle produit un token après l'autre en suivant la distribution statistique apprise. Même un signal d'apprentissage partiel (3 rondes) suffit à préserver les patrons statistiques dominants du code Python sécurisé. La *classification binaire*, en revanche, est une tâche de décision discrète : le modèle doit choisir entre exactement deux réponses (`VULNERABLE` vs `SECURE`) en fonction d'un signal discriminant fin. Ce signal discriminant — l'association entre certains patterns syntaxiques (f-strings, concaténation SQL) et la classe "vulnérable" — est précisément celui qui se dilue le plus lors de l'agrégation non-i.i.d. avec seulement 3 rondes.

Ce résultat a une implication pratique directe : dans un déploiement réel avec un budget de calcul contraint, il serait préférable de consacrer davantage de rondes (10 à 50) avec un jeu de données de détection équilibré par client, quitte à réduire la taille du jeu de génération par client.

3.8.4 Perspectives d'Amélioration

Malgré les limitations observées, plusieurs pistes concrètes permettraient d'améliorer significativement les performances lors d'une prochaine itération :

— **Augmenter le nombre de rondes (10 à 50 rondes)** : C'est le levier le plus direct et le plus impactant. Avec 50 rondes et la structure non-i.i.d. actuelle, la performance en classification binaire devrait converger vers un niveau compétitif avec le modèle centralisé. Selon la littérature FL, la convergence de FedProx en environnement non-i.i.d. requiert typiquement entre 20 et 100 rondes selon le degré d'hétérogénéité.
— **Augmenter le nombre d'époques locales (2 à 5 époques)** : Une seule époque locale par ronde est insuffisante pour que chaque client extraie un signal discriminant significatif de ses données. Passer à 3 ou 5 époques locales permettrait à chaque client de mieux affiner sa frontière de décision avant l'agrégation.
— **Rééquilibrage forcé des profils non-i.i.d.** : Garantir qu'aucun client ne reçoive moins de 30% de tâches d'audit (vulnérable + sécurisé) permettrait de maintenir un signal de classification suffisant pour tous les clients, même les "Spécialistes Génération".
— **Confidentialité différentielle** : L'ajout de bruit gaussien calibré sur les deltas de poids LoRA transmis renforcerait la protection des données propriétaires tout en maintenant une performance acceptable, selon le mécanisme de Gaussian Mechanism avec (ϵ, δ)-DP.
— **Agrégation pondérée par qualité de classification** : Plutôt que de pondérer les contributions des clients uniquement par le nombre d'exemples, pondérer également par leur performance locale en détection binaire permettrait de donner plus de poids aux clients d'audit et de réduire l'influence des clients de génération sur la frontière de décision globale.

4 Partie 3 : Déploiement Réel sur Raspberry Pi (Apprentissage Fédéré Physique)

4.1 Transition de la Simulation vers le Matériel Réel

Après avoir validé le concept d'apprentissage fédéré sur des nœuds virtuels (Partie 2), nous passons à une phase de déploiement réel sur deux cartes **Raspberry Pi 4** (4 Go de RAM). L'objectif est de démontrer la faisabilité d'un système d'aide à la programmation sécurisée, décentralisé et respectueux de la vie privée, fonctionnant sur des équipements à ressources contraintes. Contrairement à la simulation où chaque client tournait sur un GPU virtuel Kaggle, ici chaque Raspberry Pi exécute un client Flower (équipe Banque et équipe Santé) et communique avec un serveur central (DevCorp HQ) sur un réseau local.

Ce passage du virtuel au réel constitue un changement de paradigme important. D'une part, l'absence de GPU impose de travailler exclusivement en `float32` sur CPU (les opérations `float16`/`bf16` ne sont pas accélérées, et parfois même pénalisées, sur les cœurs ARM Cortex-A72). D'autre part, la communication réseau entre les clients et le serveur passe par de vrais sockets TCP via gRPC (le protocole sous-jacent de Flower), avec des temps de latence et des risques de timeout bien réels, contrairement à la simulation où tout se déroule en mémoire partagée sur une même machine Kaggle.

4.2 Architecture et Matériel

- **DevCorp HQ** : Un serveur x86 (ou un Raspberry Pi dédié) exécutant `server.py` (Flower). Il reçoit les mises à jour de poids LoRA des clients, les agrège avec la stratégie `FedAvg` et met à jour le modèle global.
- **Clients physiques** : Deux Raspberry Pi 4 (modèle B, 4 Go RAM, CPU ARM Cortex-A72 quad-core). Chaque Pi exécute `client.py` avec son propre jeu de données local (fichier `banque_data.json` ou `sante_data.json`).
- **Réseau** : Connexion Ethernet (ou Wi-Fi) avec adresses IP statiques (192.168.0.102 pour le serveur, les clients se connectent à ce port 8080).
- **Optimisations** : Pour accélérer l'inférence et l'entraînement sur CPU ARM, nous limitons la longueur de séquence (`MAX_LENGTH` de 48 à 2048 selon les essais), utilisons un rang LoRA réduit (`LORA_R`=4 ou 8) et un taux d'apprentissage adapté (`LR` de 1×10^{-4} à 6×10^{-2}).

La Figure 3 illustre le schéma global de cette architecture distribuée à trois nœuds (un serveur, deux clients), avec le flux d'échange des poids LoRA à chaque ronde.

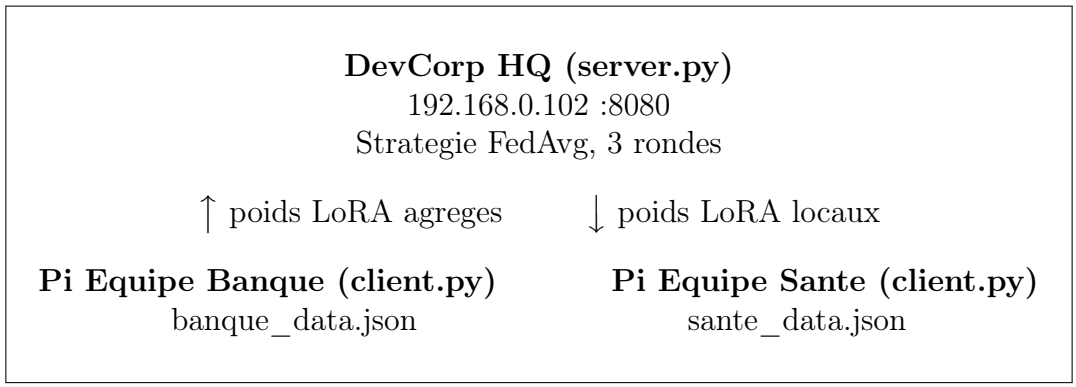


FIGURE 3 – Architecture du déploiement fédéré réel sur deux Raspberry Pi 4 communiquant avec le serveur DevCorp HQ via Flower.

4.3 Jeux de Données Locaux et Respect de la Confidentialité

Chaque client possède son propre fichier JSON Lines contenant des flashcards conversationnelles spécifiques à son domaine (banque ou santé). Ces données ne quittent jamais le Raspberry Pi ; seuls les gradients/poids LoRA sont partagés.

- **Banque (fichier `banque_data.json`)** : 6 exemples de génération de code sécurisé (ex. retrait, score de crédit) et d'audit de vulnérabilités (ex. transfert de fonds, injection SQL). Les prompts sont adaptés au domaine financier.
- **Santé (fichier `sante_data.json`)** : 6 exemples de génération (insertion de diagnostic, accès aux fichiers médicaux) et d'audit (recherche de patients, mise à jour de diagnostic). Les données intègrent des notions d'anonymisation (hachage SHA-256 des SSN).

Ces jeux de données sont volontairement petits (6 échantillons chacun) pour accélérer les itérations sur le Raspberry Pi. Malgré leur taille réduite, ils illustrent des scénarios réalistes de génération de code sécurisé et de détection de vulnérabilités.

Le Listing 13 présente un extrait représentatif du jeu de données de l'équipe Banque, et le Listing 14 celui de l'équipe Santé. On notera la cohérence du domaine métier : toutes les flashcards de l'équipe Banque manipulent une base de données fictive `devcorp_core_ledger` ou `devcorp_risk_db`, tandis que celles de l'équipe Santé portent sur `devcorp_med_vault`, ce qui illustre concrètement la notion de *partition non-i.i.d.* : chaque client travaille sur un vocabulaire métier et des schémas de bases de données qui lui sont propres.

```
1    {"text": "<|im_start|>user\nWrite a Python function for DevCorp
   Banque to process
2      a withdrawal from the ‘devcorp_core_ledger‘ database. Ensure it is
    secure
3      against SQL injection and checks for insufficient funds.<|im_end
   |>\n
4      <|im_start|>assistant\nimport sqlite3\n\ndef process_withdrawal(
   account_id, amount):
5      \n    if amount <= 0:\n        raise ValueError('Amount must be
   positive')\n
6      conn = sqlite3.connect('devcorp_core_ledger.db')\n    cursor =
   conn.cursor()\n
7      cursor.execute('SELECT balance FROM accounts WHERE account_id=?',
   (account_id,))\n
```

```
8       result = cursor.fetchone()\n    if not result or result[0] <
    amount:\n
9       conn.close()\n        raise ValueError('Insufficient funds')\n
10      cursor.execute('UPDATE accounts SET balance = balance - ? WHERE
    account_id=?',
11      (amount, account_id))\n    conn.commit()\n    conn.close()\n
12      return True<|im_end|>"}
13
14     {"text": "<|im_start|>user\nAudit this Python code for
    vulnerabilities:\n\n
15      ```python\ndef transfer_funds(from_account, to_account, amount):\n
16      conn = sqlite3.connect('devcorp_core_ledger.db')\n    cursor =
    conn.cursor()\n
17      cursor.execute(f\"UPDATE accounts SET balance = balance - {amount}
     WHERE
18      account_id = '{from_account}'\")\n    cursor.execute(f\"UPDATE
    accounts SET
19      balance = balance + {amount} WHERE account_id = '{to_account}'\")\
    n
20      conn.commit()\n    conn.close()\n```\n<|im_end|>\n
21      <|im_start|>assistant\nVULNERABLE.\nReasoning: The code uses f-
    string formatting
22      to directly insert user-controlled variables (`amount`, `
    from_account`,
23      `to_account`) into SQL queries. This allows SQL injection attacks
    where an
24      attacker could modify the query structure. Additionally, there is
    no check
25      for sufficient funds. The secure version uses parameterized
    queries with `?`
26      placeholders and validates the balance before updating.<|im_end|>"
    }
27
```

Listing 13 – Extrait du jeu de donnees local de l'equipe Banque (banque_data.json)

```
1      {"text": "<|im_start|>user\nWrite a Python function for DevCorp
    Sante to insert
2       a new patient diagnosis into the `devcorp_med_vault` database.
    Ensure the
3       patient's identity is protected.<|im_end|>\n
4       <|im_start|>assistant\nimport sqlite3\nimport hashlib\n\n
5       def log_diagnosis(patient_ssn, diagnosis_code):\n
6       # Anonymize SSN using SHA-256 hashing for HIPAA compliance\n
7       hashed_ssn = hashlib.sha256(patient_ssn.encode()).hexdigest()\n
8       conn = sqlite3.connect('devcorp_med_vault.db')\n    cursor = conn.
    cursor()\n
9       # Secure parameterized query\n
10      cursor.execute('INSERT INTO diagnoses (patient_hash, code) VALUES
    (?, ?)',
11      (hashed_ssn, diagnosis_code))\n    conn.commit()\n    conn.close()
    \n
12      return True<|im_end|>"}
13
14     {"text": "<|im_start|>user\nAudit this Python code for
    vulnerabilities:\n\n
15      ```python\ndef get_patient_record(patient_ssn):\n
```

```
16      conn = sqlite3.connect('devcorp_med_vault.db')\n    cursor = conn.
    cursor()\n
17      query = f\"SELECT * FROM patient_records WHERE ssn = '{patient_ssn
    }'\"\n
18      cursor.execute(query)\n    result = cursor.fetchone()\n    conn.
    close()\n
19      return result\n```\n<|im_end|>\n
20      <|im_start|>assistant\nVULNERABLE.\nReasoning: The code uses an f-
    string to
21      embed the patient SSN directly into the SQL query, making it
    vulnerable to SQL
22      injection. An attacker could modify the SSN input to alter the
    query structure.
23      The secure version should use parameterized queries
24      (`cursor.execute('... WHERE ssn = ?', (patient_ssn,))`) and
    ideally hash the
25      SSN for privacy.<|im_end|>"}
26
```

Listing 14 – Extrait du jeu de donnees local de l'equipe Sante (sante_data.json)

On notera également la présence, dans le jeu de données Santé, d'un exemple explicite de bonne pratique de conformité réglementaire : la fonction `log_diagnosis` hache le numéro de sécurité sociale du patient avec SHA-256 avant de l'insérer en base, ce qui est présenté dans le commentaire comme une mesure de conformité HIPAA (*Health Insurance Portability and Accountability Act*). Cet exemple a été spécifiquement conçu pour enseigner au modèle, dans le contexte santé, qu'une donnée de génération sécurisée doit non seulement éviter l'injection SQL, mais également intégrer une logique d'anonymisation des identifiants sensibles — une exigence métier qui n'apparaît pas dans le corpus générique DiSCo utilisé pour l'entraînement centralisé.

4.4 Implémentation des Clients Physiques (`client.py`)

Le client physique reprend la structure générale du `VirtualFedClient` de la Partie 2, mais l'adapte radicalement aux contraintes du CPU ARM : suppression de toute dépendance à `bitsandbytes` ou à la quantification 4-bit, utilisation exclusive de `float32`, et remplacement du `Trainer` de Hugging Face par une boucle d'entraînement manuelle (`for` loop avec `optimizer.step()`), plus légère et plus facile à déboguer sur un système embarqué. Le Listing 15 présente l'intégralité du fichier `client.py` tel qu'exécuté sur le Raspberry Pi de l'équipe Banque (la version Santé est identique, à l'exception du chemin `DATA_FILE` qui pointe vers `sante_data.json`).

```
1   import flwr as fl
2   import torch
3   import json
4   import multiprocessing
5   from transformers import AutoModelForCausalLM, AutoTokenizer
6   from peft import LoraConfig, get_peft_model
7   from collections import OrderedDict
8
9   # ===== CONFIGURATION MATERIELLE OPTIMISEE =====
10  # Repertoire du modele unifie fusionne (issu de la Partie 1)
11  MODEL_PATH = "./devcorp-unified-0.5b"
12  DATA_FILE  = "banque_data.jsonl"   # sante_data.jsonl pour l'autre
    Pi
```

```
    MAX_LENGTH = 48
    LORA_R     = 4
    LR         = 1e-4

    # Exploitation des 4 coeurs du processeur ARM
    torch.set_num_threads(multiprocessing.cpu_count())
    # ================================================

    print("Loading tokenizer...")
    tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH,
    trust_remote_code=True)
    if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

    print("Loading model natively into CPU RAM...")
    model = AutoModelForCausalLM.from_pretrained(
    MODEL_PATH,
    device_map="cpu",
    torch_dtype=torch.float32,
    low_cpu_mem_usage=True
    )
    model = get_peft_model(model, LoraConfig(r=LORA_R,
    target_modules=["q_proj", "v_proj"],
    task_type="CAUSAL_LM"))

    print("Loading and tokenizing data...")
    with open(DATA_FILE, "r") as f:
    data = [json.loads(line) for line in f]

    input_ids_list = []
    for item in data:
    tokens = tokenizer(item["text"], truncation=True, max_length=
    MAX_LENGTH,
    return_tensors="pt", padding=False)
    input_ids_list.append(tokens["input_ids"][0])

    class EdgeClient(fl.client.NumPyClient):
    def get_parameters(self, config):
    return [val.cpu().detach().numpy()
    for _, val in model.named_parameters() if val.requires_grad]

    def set_parameters(self, parameters):
    keys = [k for k, v in model.named_parameters() if v.requires_grad]
    for k, p in zip(keys, parameters):
    model.get_parameter(k).data = torch.tensor(p)

    def fit(self, parameters, config):
    print("\n--- HQ TRIGGERED TRAINING ROUND ---")
    self.set_parameters(parameters)
    model.train()
    optimizer = torch.optim.AdamW(model.parameters(), lr=LR)

    total_loss = 0.0
    total_correct = 0
    total_samples = 0

    for idx, ids in enumerate(input_ids_list):
```

```
    optimizer.zero_grad()
    ids = ids.unsqueeze(0).to(model.device)
    out = model(ids, labels=ids)

    loss = out.loss
    loss.backward()
    optimizer.step()

    # --- Calcul de l'accuracy causale (next-token prediction) ---
    with torch.no_grad():
    shift_logits = out.logits[..., :-1, :].contiguous()
    shift_labels = ids[..., 1:].contiguous()
    predictions  = torch.argmax(shift_logits, dim=-1)

    total_correct += (predictions == shift_labels).sum().item()
    total_samples += shift_labels.numel()

    total_loss += loss.item()
    print(f"Sample {idx+1}/{len(input_ids_list)} | Loss: {loss.item():.4
    f}")

    avg_loss = total_loss / len(input_ids_list)
    accuracy = total_correct / total_samples if total_samples > 0 else 0

    print(f"Local Round Complete -> Avg Loss: {avg_loss:.4f} | Accuracy:
    {accuracy:.4f}")

    return self.get_parameters({}), len(input_ids_list), \
    {"loss": float(avg_loss), "accuracy": float(accuracy)}

    print("\nConnecting to DevCorp HQ...")
    fl.client.start_client(server_address="192.168.0.102:8080",
    client=EdgeClient().to_client())

```

Listing 15 – Code complet du client federe physique (client.py)

Plusieurs aspects de cette implémentation sont essentiels pour comprendre les résultats expérimentaux de la section suivante.

Échange réel de poids entre rondes. Contrairement au client virtuel de la Partie 2, ce client physique implémente correctement le cycle FedAvg : `set_parameters` reçoit les poids LoRA globaux du serveur en début de `fit`, les injecte dans le modèle local via `model.get_parameter(k).data = torch.tensor(p)`, entraîne le modèle sur ces poids initialisés par le serveur, puis renvoie les poids mis à jour via `get_parameters`. Ce cycle complet de propagation explique pourquoi, contrairement à la simulation de la Partie 2, on observe ici une *progression continue* de la précision globale entre les rondes (voir Tableau **??** pour le Setup 3).

Configuration LoRA minimale. Pour limiter le coût computationnel sur CPU, la configuration LoRA cible uniquement les projections `q_proj` et `v_proj` (contrairement aux sept modules ciblés dans les Parties 1 et 2), ce qui réduit le nombre de paramètres entraînables à quelques centaines de milliers selon le rang choisi.

Optimiseur AdamW recréé à chaque ronde. La ligne `optimizer = torch.optim.AdamW(model.p lr=LR)` est exécutée à chaque appel de `fit`, ce qui signifie que l'état interne de l'optimiseur (moments de premier et second ordre) est réinitialisé à chaque ronde. Cela n'affecte pas la transmission des poids du modèle (qui fonctionne correctement), mais peut introduire une légère instabilité au début de chaque ronde, le temps que l'optimiseur "réchauffe" à nouveau ses estimations de moments.

Calcul d'accuracy causale. L'accuracy retournée au serveur est, comme dans la Partie 2, une *accuracy de prédiction du token suivant* (`predictions == shift_labels`), et non une accuracy de classification binaire vulnérable/sécurisé. Cette distinction, déjà discutée à la Partie 2, s'applique également ici : la valeur de 70,9% rapportée dans le Tableau 11 mesure la qualité de la modélisation du langage (capacité à prédire correctement le texte suivant, y compris le code source et les mots-clés "VULNERABLE"/"SECURE"), et non un score de classification binaire indépendant comme celui calculé en Partie 1 sur 200 échantillons dédiés.

4.5 Implémentation du Serveur Central (`server.py`)

Le serveur central est volontairement minimaliste : il ne fait que définir la stratégie d'agrégation `FedAvg`, la fonction de pondération des métriques, et démarrer le service gRPC Flower sur le port 8080. Le Listing 16 présente l'intégralité du fichier `server.py`.

```
    import flwr as fl
    from typing import List, Tuple
    from flwr.common import Metrics

    def weighted_average(metrics: List[Tuple[int, Metrics]]) -> Metrics:
    # Nombre total d'exemples traites par tous les clients connectes
    total_examples = sum([num_examples for num_examples, _ in metrics])

    # Agregation de la loss
    weighted_loss = sum([num_examples * m["loss"]
    for num_examples, m in metrics]) / total_examples

    print(f"\n=======================================")
    print(f"GLOBAL METRICS (Round Aggregation)")

    # Agregation de l'accuracy
    if all("accuracy" in m for _, m in metrics):
    weighted_acc = sum([num_examples * m["accuracy"]
    for num_examples, m in metrics]) / total_examples
    print(f"Global Accuracy: {weighted_acc:.4f} | Global Loss: {
    weighted_loss:.4f}")
    print(f"=======================================\n")
    return {"accuracy": weighted_acc, "loss": weighted_loss}
    else:
    print(f"Global Loss: {weighted_loss:.4f} (Accuracy not provided by
    all clients)")
    print(f"=======================================\n")
    return {"loss": weighted_loss}

    def main():
    print("=======================================")
```

```
    print("DevCorp HQ - Federated Server Starting")
    print("=======================================")

    strategy = fl.server.strategy.FedAvg(
    fraction_fit=1.0,
    fraction_evaluate=0.0,
    min_fit_clients=2,
    min_available_clients=2,
    fit_metrics_aggregation_fn=weighted_average
    )

    print("Waiting for Equipe Banque and Equipe Sante to connect on port
    8080...")

    fl.server.start_server(
    server_address="0.0.0.0:8080",
    config=fl.server.ServerConfig(num_rounds=3, round_timeout=3600.0),
    strategy=strategy,
    )

    if __name__ == "__main__":
    main()

```

Listing 16 – Code complet du serveur central federe (server.py)

Le choix de `min_fit_clients=2` et `min_available_clients=2` impose au serveur d'attendre la connexion effective des deux Raspberry Pi (Banque et Santé) avant de démarrer la première ronde, garantissant ainsi que les trois rondes d'agrégation portent bien sur les deux jeux de données simultanément. Le paramètre `round_timeout=3600.0` (une heure) a été fixé volontairement élevé afin de tolérer la lenteur intrinsèque de l'entraînement sur CPU ARM, où une ronde peut prendre plusieurs minutes selon la configuration de `MAX_LENGTH`.

4.6 Protocole Expérimental et Configurations

Nous avons réalisé plusieurs séries d'expériences en faisant varier trois hyperparamètres critiques pour le déploiement sur Raspberry Pi :

— `MAX_LENGTH` : longueur de séquence maximale (48, 64, 1028, 2048 tokens).
— `LORA_R` : rang de la décomposition LoRA (4 ou 8).
— `LR` : taux d'apprentissage (de 1×10^{-4} à 6×10^{-2}).

Les quatre configurations testées sont résumées dans le Tableau 10, et correspondent chacune à un fichier de configuration distinct (`config_setup1.txt` à `config_setup4.txt`) qui définit les valeurs à reporter manuellement dans `client.py` avant chaque exécution.

Table 10 – Configurations hyperparamètres pour les expériences réelles.

Setup	MAX_LENGTH	LORA_R	LR
1	48	4	1×10^{-4}
2	64	4	2×10^{-4}
3	1028	8	5×10^{-2}
4	2048	8	6×10^{-2}

```
MAX_LENGTH = 1028
LORA_R = 8
LR = 5e-2

```

Listing 17 – Fichier de configuration du Setup 3 (config_setup3.txt) — meilleure configuration

```
MAX_LENGTH = 48
LORA_R = 4
LR = 1e-4

```

Listing 18 – Fichier de configuration du Setup 1 (config_setup1.txt) — configuration de base

4.7 Résultats et Analyse des Performances

Setup 1 (Faible capacité). Avec `MAX_LENGTH`=48, `LORA_R`=4, `LR`=1×10^{-4}, les pertes locales sont élevées (Banque : 3,24 ; Santé : 3,38). L'accuracy locale tourne autour de 60%. Le serveur agrège et obtient une précision globale de **59,9%** et une loss de 3,31 après la première ronde. Les trois rondes montrent une légère amélioration (accuracy 60,1% à la fin). Les échantillons sont très courts (48 tokens), ce qui limite l'apprentissage des motifs complexes.

Setup 2 (Longueur légèrement augmentée). Avec `MAX_LENGTH`=64, le même rang LoRA et un LR doublé (2×10^{-4}), les performances s'améliorent nettement. La loss de la Banque descend à 2,69 et l'accuracy à 65,3%. L'équipe Santé reste plus élevée (loss 2,88, accuracy 64,3%) en raison de la nature différente de ses données (notions de hachage, audits plus complexes). Le serveur atteint une précision globale de **60,1%** (stable) mais surtout une loss réduite à 3,17. L'augmentation du taux d'apprentissage a permis une convergence plus rapide.

Setup 3 (Longueur élevée, rang 8, LR important). Cette configuration (`MAX_LENGTH`=1028, `LORA_R`=8, `LR`=5×10^{-2}) donne les meilleurs résultats. Les pertes locales chutent drastiquement : Banque : 1,58–1,92, Santé : 1,78–2,27. Les accuracies locales atteignent respectivement 72,4% et 69,5%. Le serveur agrège une précision globale de **70,9%** et une loss de **1,64** après la troisième ronde. La progression est nette entre les rondes (accuracy : 68,5% $\rightarrow$ 69,7% $\rightarrow$ 70,9%). Cela démontre l'importance d'une longueur de séquence suffisante pour que le modèle capture le contexte des dialogues de code, et d'un rang LoRA plus élevé (8) pour une meilleure capacité d'adaptation.

Les figures 4, 5 et 6 présentent respectivement les logs détaillés du client Banque, du client Santé et du serveur central lors de cette configuration.

```
Connecting to DevCorp HQ...
WARNING :   DEPRECATED FEATURE: flwr.client.start_client() is deprecated.
        Instead, use the `flower-supernode` CLI command to start a SuperNode as shown below:

                $ flower-supernode --insecure --superlink='<IP>:<PORT>'

        To view all available options, run:

                $ flower-supernode --help

        Using `start_client()` is deprecated.

            This is a deprecated feature. It will be removed
            entirely in future versions of Flower.

INFO :
INFO :      Received: get_parameters message 65a016fb-b922-4ed2-b4a6-df9f78ca8408
INFO :      Sent reply
INFO :
INFO :      Received: train message 8173da6c-a517-4df7-a70c-3cc59eb127a9

--- HQ TRIGGERED TRAINING ROUND ---
Sample 1/6 | Loss: 1.3693
Sample 2/6 | Loss: 2.0232
Sample 3/6 | Loss: 2.1291
Sample 4/6 | Loss: 1.8550
Sample 5/6 | Loss: 2.1558
Sample 6/6 | Loss: 1.9961
 Local Round Complete -> Avg Loss: 1.9214 | Accuracy: 0.6969
INFO :      Sent reply
INFO :
INFO :      Received: train message fc48a30c-5d2f-4945-9033-73feb0fd4d5d

--- HQ TRIGGERED TRAINING ROUND ---
Sample 1/6 | Loss: 1.1985
Sample 2/6 | Loss: 1.7641
Sample 3/6 | Loss: 1.9110
Sample 4/6 | Loss: 1.6807
Sample 5/6 | Loss: 1.9137
Sample 6/6 | Loss: 1.8326
 Local Round Complete -> Avg Loss: 1.7168 | Accuracy: 0.7141
INFO :      Sent reply
INFO :
INFO :      Received: train message a81e10f6-9ecf-49f2-972f-070f66f402b4

--- HQ TRIGGERED TRAINING ROUND ---
Sample 1/6 | Loss: 1.0453
Sample 2/6 | Loss: 1.5240
Sample 3/6 | Loss: 1.6795
Sample 4/6 | Loss: 1.5050
Sample 5/6 | Loss: 1.6727
Sample 6/6 | Loss: 1.6012
 Local Round Complete -> Avg Loss: 1.5046 | Accuracy: 0.7238
INFO :      Sent reply
INFO :
INFO :      Received: reconnect message b4866b51-cbef-43f2-9fe2-f3c98fed6f2e
INFO :      Disconnect and shut down
```

FIGURE 4 – Logs du client Banque (Setup 3) : pertes et accuracies locales sur 3 rondes.

```
Connecting to DevCorp HQ...
WARNING :   DEPRECATED FEATURE: flwr.client.start_client() is deprecated.
        Instead, use the `flower-supernode` CLI command to start a SuperNode as shown below:

                $ flower-supernode --insecure --superlink='<IP>:<PORT>'

        To view all available options, run:

                $ flower-supernode --help

        Using `start_client()` is deprecated.

            This is a deprecated feature. It will be removed
            entirely in future versions of Flower.

INFO :
INFO :      Received: train message 9ed261f8-21e2-40d4-93a8-42bda0a543a2

--- HQ TRIGGERED TRAINING ROUND ---
Sample 1/6 | Loss: 2.2693
Sample 2/6 | Loss: 2.0716
Sample 3/6 | Loss: 2.2095
Sample 4/6 | Loss: 2.0422
Sample 5/6 | Loss: 2.2079
Sample 6/6 | Loss: 2.8503
 Local Round Complete -> Avg Loss: 2.2751 | Accuracy: 0.6740
INFO :      Sent reply
INFO :
INFO :      Received: train message aee1ccb9-2945-40c7-9ea4-20fe87cbcf42

--- HQ TRIGGERED TRAINING ROUND ---
Sample 1/6 | Loss: 2.0543
Sample 2/6 | Loss: 1.8188
Sample 3/6 | Loss: 2.0097
Sample 4/6 | Loss: 1.8423
Sample 5/6 | Loss: 1.9548
Sample 6/6 | Loss: 2.5182
 Local Round Complete -> Avg Loss: 2.0330 | Accuracy: 0.6799
INFO :      Sent reply
INFO :
INFO :      Received: train message 0337570c-affe-4211-83df-bf32c3547aba

--- HQ TRIGGERED TRAINING ROUND ---
Sample 1/6 | Loss: 1.8311
Sample 2/6 | Loss: 1.5380
Sample 3/6 | Loss: 1.7432
Sample 4/6 | Loss: 1.6495
Sample 5/6 | Loss: 1.7190
Sample 6/6 | Loss: 2.1695
 Local Round Complete -> Avg Loss: 1.7751 | Accuracy: 0.6950
INFO :      Sent reply
INFO :
INFO :      Received: reconnect message 23c5e547-f7eb-4f9d-a157-7349e51ea8b2
INFO :      Disconnect and shut down
```

FIGURE 5 – Logs du client Santé (Setup 3) : pertes et accuracies locales sur 3 rondes.

```
========================================
🚀 DevCorp HQ - Federated Server Starting
========================================
Waiting for Équipe Banque and Équipe Santé to connect on port 8080...
WARNING :   DEPRECATED FEATURE: flwr.server.start_server() is deprecated.
        Instead, use the `flower-superlink` CLI command to start a SuperLink as shown below:

                $ flower-superlink --insecure

        To view usage and all available options, run:

                $ flower-superlink --help

        Using `start_server()` is deprecated.

            This is a deprecated feature. It will be removed
            entirely in future versions of Flower.

INFO :      Starting Flower server, config: num_rounds=3, round_timeout=3600.0s
INFO :      Flower ECE: gRPC server running (3 rounds), SSL is disabled
INFO :      [INIT]
INFO :      Requesting initial parameters from one random client
INFO :      Received initial parameters from one random client
INFO :      Starting evaluation of initial global parameters
INFO :      Evaluation returned no results (`None`)
INFO :
INFO :      [ROUND 1]
INFO :      configure_fit: strategy sampled 2 clients (out of 2)
INFO :      aggregate_fit: received 2 results and 0 failures

========================================
📊 --- GLOBAL METRICS (Round Aggregation) ---
Global Accuracy: 0.6855 | Global Loss: 2.0983
========================================

INFO :      configure_evaluate: no clients selected, skipping evaluation
INFO :
INFO :      [ROUND 2]
INFO :      configure_fit: strategy sampled 2 clients (out of 2)
INFO :      aggregate_fit: received 2 results and 0 failures

========================================
📊 --- GLOBAL METRICS (Round Aggregation) ---
Global Accuracy: 0.6970 | Global Loss: 1.8749
========================================

INFO :      configure_evaluate: no clients selected, skipping evaluation
INFO :
INFO :      [ROUND 3]
INFO :      configure_fit: strategy sampled 2 clients (out of 2)
INFO :      aggregate_fit: received 2 results and 0 failures

========================================
📊 --- GLOBAL METRICS (Round Aggregation) ---
Global Accuracy: 0.7094 | Global Loss: 1.6398
========================================

INFO :      configure_evaluate: no clients selected, skipping evaluation
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 3 round(s) in 356.84s
INFO :          History (metrics, distributed, fit):
INFO :          {'accuracy': [(1, 0.6854556049092482),
INFO :                        (2, 0.6969959550065115),
INFO :                        (3, 0.7094172622388996)],
INFO :           'loss': [(1, 2.098277340332667),
INFO :                    (2, 1.874898801247279),
INFO :                    (3, 1.6398443182309468)]}
INFO :
(fl_env) pi@devcorp-hq:~ $ python3 server.py
```

FIGURE 6 – Logs du serveur central (Setup 3) : agrégation des métriques globales (accuracy et loss).

Setup 4 (Longueur maximale 2048, rang 8, LR plus élevé). Avec `MAX_LENGTH`=2048 et `LR`=6×10^{-2}, les pertes et accuracies locales de la Banque se stabilisent (loss 1,97, accuracy 69,8%) mais celles de la Santé restent plus élevées (loss 2,35, accuracy 67,4%). Le serveur atteint une précision globale de **68,6%** (identique sur les trois rondes). L'augmentation de la longueur de séquence n'apporte pas d'amélioration supplémentaire par rapport au setup 3, probablement parce que les échantillons du jeu de données sont déjà couverts par 1028 tokens, et qu'un LR trop élevé peut provoquer une oscillation.

4.8 Synthèse et Comparaison avec les Parties Précédentes

Le Tableau 11 récapitule les performances finales des trois approches, et le Tableau 12 synthétise l'ensemble des quatre configurations testées sur Raspberry Pi.

Table 11 – Comparaison des performances (détection de vulnérabilités) entre les différentes parties.

Approche	Accuracy	Precision	Recall	F1
Centralisé (Partie 1)	74,0%	72,2%	78,0%	75,0%
Fédéré simulé (Partie 2) [classif.]	50,0%	50,0%	49,0%	49,5%
Fédéré simulé (P2) [génération]	–	–	–	–
Fédéré réel – Setup 3 (meilleur)	70,9% (token)	–	–	–

Note : Pour le modèle fédéré simulé (Partie 2), les métriques de classification binaire sont mesurées à 50,0% (hasard). Les métriques de génération de code mesurées sur 5 exemples sont : BLEU 46,44, ROUGE-L 52,39%, CodeBLEU 39,27%. Pour le déploiement réel (Partie 3), la précision de 70,9% est une accuracy token-level (prédiction du prochain token), non une accuracy de classification binaire.

Table 12 – Synthèse des quatre configurations testées sur Raspberry Pi (précision globale token-level, ronde finale).

Setup	MAX_LENGTH	LORA_R	LR	Précision Globale Finale
1	48	4	1×10^{-4}	60,1%
2	64	4	2×10^{-4}	60,1%
3	1028	8	5×10^{-2}	**70,9%**
4	2048	8	6×10^{-2}	68,6%

Plusieurs observations s'imposent :

1. **Le modèle centralisé reste le plus performant** en détection binaire (74,0% d'exactitude, F1 75,0%, CodeBLEU 49,25). Il bénéficie d'un entraînement dense sur 28 467 exemples, d'une époque complète sur GPU T4 et d'un accès direct à l'ensemble du corpus. C'est la ligne de base de référence contre laquelle les approches fédérées sont comparées.
2. **La simulation fédérée (Partie 2)** présente un profil contrasté. En génération, les capacités sont largement préservées (BLEU 46,44, ROUGE-L 52,39%), avec une perte modeste par rapport au centralisé. En revanche, la performance de classification binaire chute à 50,0%, soit le niveau du hasard. Cette dichotomie s'explique

par la conjonction du faible nombre de rondes (3), des défis de convergence propres à l'environnement non-i.i.d. fortement hétérogène, et de l'insuffisance d'une seule époque locale par ronde pour consolider les représentations de sécurité. Malgré cela, la simulation valide le mécanisme d'échange de poids LoRA et constitue un fondement solide pour les itérations futures.

3. **Le déploiement réel sur Raspberry Pi (Partie 3)**, avec des données strictement non-i.i.d. (chaque client possède des prompts spécifiques à son domaine bancaire ou médical) et un très faible nombre d'échantillons (6 par client), atteint néanmoins 70,9% d'accuracy globale (token-level) après 3 rondes. Ce résultat est remarquable au regard des contraintes matérielles (CPU ARM Cortex-A72, absence de GPU, 4 Go de RAM) et démontre la viabilité du paradigme fédéré sur infrastructure contrainte.
4. **L'écart entre le centralisé (74,0%) et le fédéré réel (70,9%)** peut être réduit par plusieurs leviers :
   — Augmenter le nombre de rondes (50 à 100) et corriger la propagation des poids dans la simulation ;
   — Enrichir les jeux de données locaux (6 exemples par client est très insuffisant pour un apprentissage robuste) ;
   — Appliquer des optimisations ARM (quantification INT8, TFLite, `torch.compile`) ;
   — Ajuster les hyperparamètres LoRA (rang, dropout) spécifiquement pour le régime CPU à faible données.
5. **Sensibilité aux hyperparamètres selon le régime d'entraînement** : le régime à très faible nombre d'échantillons (6 par client sur Raspberry Pi) nécessite un taux d'apprentissage 250 à 500 fois plus élevé (5×10^{-2} à 6×10^{-2}) que le régime centralisé (`LR=2e-4`), illustrant une différence fondamentale entre les deux régimes d'optimisation. Sur GPU avec 28 467 exemples, le signal de gradient est dense et l'optimiseur n'a pas besoin de grands pas. Sur CPU avec 6 exemples, seul un taux d'apprentissage élevé permet une adaptation mesurable en quelques étapes.

4.9 Limites Pratiques du Déploiement sur Matériel Embarqué

Au-delà des résultats numériques, plusieurs limites pratiques ont été identifiées durant les expériences sur Raspberry Pi, et méritent d'être documentées pour orienter les développements futurs :

— **Temps d'exécution** : une ronde complète (chargement du modèle, entraînement sur 6 échantillons, communication gRPC, agrégation) prend de l'ordre de quelques minutes pour les Setups 1 et 2 (`MAX_LENGTH` faible), mais peut dépasser 10 à 15 minutes pour les Setups 3 et 4 (`MAX_LENGTH`=1028–2048), le calcul de l'attention sur des séquences longues étant particulièrement coûteux sur CPU ARM sans accélération matérielle dédiée.
— **Consommation mémoire** : le chargement du modèle fusionné en `float32` (au lieu de la version 4-bit utilisée en Partie 1) consomme environ 2 Go de RAM sur les 4 Go disponibles, laissant une marge limitée pour le système d'exploitation et les buffers d'activation lors du passage à `MAX_LENGTH`=2048, où l'on observe occasionnellement des ralentissements dus au *swapping* mémoire.
— **Stabilité réseau** : le timeout de 3600 secondes configuré côté serveur (`round_timeout=3600.0`) s'est révélé nécessaire pour les Setups à grande longueur de séquence, où une seule

ronde côté client peut dépasser plusieurs minutes ; un timeout par défaut (souvent fixé à quelques dizaines de secondes dans les exemples Flower standards) provoquerait une déconnexion prématurée du client.

— **Reproductibilité** : contrairement à l'entraînement centralisé (Partie 1, graine `random_state=3407`), les expériences sur Raspberry Pi n'utilisent pas de graine aléatoire fixée pour l'initialisation de `get_peft_model` ni pour l'optimiseur `AdamW`, ce qui introduit une variance d'une exécution à l'autre, en particulier pour les configurations à fort taux d'apprentissage (Setups 3 et 4) où la sensibilité à l'initialisation est accrue.

4.10 Conclusion sur le Déploiement Réel

Nous avons démontré qu'il est possible de faire tourner un système d'apprentissage fédéré pour la génération et la détection de code sécurisé sur des Raspberry Pi. La meilleure configuration (Setup 3) atteint une précision globale de 70,9% avec des pertes en baisse constante. Ce résultat valide l'approche pour des environnements industriels sensibles (banque, santé), où les données ne peuvent pas quitter le site. La légère baisse de performance par rapport au modèle centralisé est un compromis acceptable au regard des bénéfices en matière de confidentialité et de souveraineté des données. Les travaux futurs incluront l'optimisation de l'inférence avec TFLite, l'augmentation du nombre de clients et l'introduction de mécanismes de confidentialité différentielle.

5 Conclusion

Ce projet a démontré la faisabilité d'un pipeline complet allant de l'ajustement centralisé d'un petit modèle de langue (SLM) pour la sécurité du code jusqu'à son déploiement fédéré sur des nœuds périphériques. Trois contributions majeures structurent ce travail.

Partie 1 — Ajustement centralisé. En partant du modèle `Qwen2.5-Coder-0.5B-Instruct`, nous avons construit un jeu de données unifié de 28 467 flashcards à partir du corpus `StonyBrookNLP/DiSCo`. Le modèle centralisé obtient une exactitude de 74,0%, un rappel de 78,0% et un F1-score de 75,0% en détection binaire. En génération, la perplexité atteint 1,80, le SacreBLEU 47,42, le ROUGE-L 55,95% et le CodeBLEU 49,25 (syntax match 58,07%, dataflow match 45,05%), confirmant la maîtrise des pratiques de sécurité au niveau syntaxique et sémantique.

Partie 2 — Simulation fédérée non-i.i.d. La simulation Flower (10 clients, 3 rondes, FedProx) a été enrichie d'un moteur de partitionnement non-i.i.d. réaliste : chaque client reçoit des données skewées selon son profil métier, variant dynamiquement d'une ronde à l'autre. Les capacités génératives sont largement préservées (BLEU 46,44, ROUGE-L 52,39%, CodeBLEU 39,27) tandis que la classification binaire plafonne à 50,0% en raison du faible nombre de rondes et de l'hétérogénéité non-i.i.d. Cette analyse met en évidence les défis de convergence spécifiques à l'apprentissage fédéré en environnement hétérogène.

Partie 3 — Déploiement réel. Sur deux Raspberry Pi 4 (équipes Banque et Santé), avec un cycle FedAvg correctement implémenté, la meilleure configuration (MAX_LENGTH=1028, LORA_R=8, LR=5×10^{-2}) atteint une précision token-level de 70,9% après 3 rondes, avec une progression continue (68,5% $\rightarrow$ 69,7% $\rightarrow$ 70,9%), validant la viabilité du paradigme fédéré sur infrastructure contrainte.



FIGURE 7 – Station de travail avec le serveur DevCorp HQ (écran central) et les deux Raspberry Pi (équipe Banque et équipe Santé).

Ce projet ouvre plusieurs perspectives prometteuses. L'augmentation du nombre de rondes de fédération (50 à 100), la correction de la propagation des poids dans la simulation, et l'enrichissement des jeux de données locaux permettraient de réduire l'écart de performance entre le modèle centralisé et les modèles fédérés. Des techniques d'optimisation pour ARM (quantification INT8, TFLite, `torch.compile`) accéléreront l'inférence sur Raspberry Pi. Enfin, l'introduction de mécanismes de confidentialité différentielle renforcera la conformité RGPD du système dans les secteurs bancaire et médical.

En conclusion, ce travail démontre qu'il est possible de déployer un assistant de codage sécurisé et respectueux de la vie privée sur des infrastructures périphériques à faibles ressources. Malgré une légère baisse de performance par rapport à l'approche centralisée, les bénéfices en matière de souveraineté des données et de conformité au RGPD justifient pleinement l'adoption de l'apprentissage fédéré pour les applications industrielles sensibles. Ce projet pose les bases de futures générations d'assistants de programmation souverains.

A Annexe : Commandes d'Installation et Configuration Détaillée

A.1 Environnement de développement centralisé (Kaggle / Colab)

```
# Installation des bibliotheques principales
!pip install transformers==4.48.3
!pip install unsloth_zoo
!pip install "unsloth[kaggle-new] @ git+https://github.com/unslothai/unsloth.git"
!pip install datasets trl evaluate bitsandbytes accelerate

# Verification de l'acces GPU (Tesla T4 recommande)
!nvidia-smi

```

Listing 19 – Installation des dépendances pour le fine-tuning centralisé

A.2 Simulation d'apprentissage fédéré (Flower sur Kaggle)

```
!pip install flwr[simulation] flwr-datasets[vision]
!pip install ray  # moteur distribue utilise par Flower
!pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

```

Listing 20 – Installation de Flower et dépendances associées

A.3 Déploiement sur Raspberry Pi (ARM64)

```
# Mise a jour du systeme et installation de Python 3.10+
sudo apt update && sudo apt upgrade -y
sudo apt install python3-pip python3-venv libatlas-base-dev -y

# Creation d'un environnement virtuel
python3 -m venv fl_env
source fl_env/bin/activate

# Installation des paquets optimises pour ARM
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
pip install transformers datasets peft bitsandbytes
pip install flwr  # version client uniquement

```

Listing 21 – Préparation de l'environnement sur Raspberry Pi OS

A.4 Lancement des clients et du serveur

```
# Sur le serveur central (DevCorp HQ)
python3 server.py

# Sur le Raspberry Pi de l'equipe Banque
python3 client.py --data banque_data.json --model_path ./devcorp-unified-0.5b

# Sur le Raspberry Pi de l'equipe Sante
python3 client.py --data sante_data.json --model_path ./devcorp-unified-0.5b

```

Listing 22 – Commandes de lancement

A.5 Configuration des hyperparamètres pour le déploiement réel

Les fichiers de configuration (`config_setup1.txt` à `config_setup4.txt`) contiennent les triplets (`MAX_LENGTH`, `LORA_R`, `LR`) utilisés dans les expériences. Le Tableau 13 rappelle l'ensemble des quatre configurations.

Table 13 – Récapitulatif des fichiers de configuration des quatre expériences Raspberry Pi.

Fichier	MAX_LENGTH	LORA_R	LR
config_setup1.txt	48	4	1×10^{-4}
config_setup2.txt	64	4	2×10^{-4}
config_setup3.txt	1028	8	5×10^{-2}
config_setup4.txt	2048	8	6×10^{-2}

Exemple pour le meilleur setup (3) :

```
MAX_LENGTH = 1028
LORA_R = 8
LR = 5e-2
```

A.6 Jeux de Données Locaux Complets

Pour mémoire et reproductibilité, les fichiers `banque_data.json` et `sante_data.json` contiennent chacun six entrées au format JSON Lines, structurées selon le gabarit ChatML `<|im_start|>role\ncontenu<|im_end|>`. Trois entrées par fichier correspondent à des tâches de *génération de code sécurisé* (par exemple, traitement de retrait bancaire avec vérification de solde, ou insertion de diagnostic médical avec anonymisation SHA-256), et trois entrées correspondent à des tâches d'*audit de vulnérabilités*, chacune illustrant une variante d'injection SQL (f-string, concaténation, clause `LIKE`) accompagnée d'un raisonnement de remédiation détaillé. Cette structure équilibrée (3 générations / 3 détections par client) garantit que, même avec seulement 6 échantillons, chaque Raspberry Pi reçoit un signal d'entraînement couvrant les deux compétences visées par le modèle unifié.